

01-PRD

01. PRD LITE — MVP LEARNING VERSION

AI T-Shirt Customization & Virtual Try-On Platform

Version: 3.0

Purpose: Demo cơ bản + học Backend System Design + Kafka

Backend: Java Spring Boot + C# .NET 8

Frontend: React + TypeScript + Fabric.js

Database: PostgreSQL

Messaging: Kafka

1. Product Goal

Xây dựng một nền tảng demo cho phép khách hàng:

- Xem sản phẩm áo thun
- Tự thiết kế áo bằng drag & drop
- Lưu thiết kế
- Gửi yêu cầu AI Virtual Try-On
- Xem ảnh preview Try-On
- Đặt hàng
- Thanh toán giả lập
- Theo dõi trạng thái đơn hàng
- Gửi feedback sau khi đơn hoàn tất
- Xem About Us

Mục tiêu chính của MVP không phải là bán hàng thật, mà là chứng minh được:

- Flow sản phẩm end-to-end
 - Backend chia service rõ ràng
 - Java + C# có thể phối hợp
 - Kafka xử lý event giữa các service
 - Database thiết kế vừa đủ, không quá phức tạp
-

2. MVP Scope

2.1 In Scope

MVP làm các chức năng sau:

Authentication

- Register
- Login
- Logout
- Refresh Token
- Role-Based Access Control

Role

MVP chỉ dùng 3 role:

- ADMIN
- STAFF
- CUSTOMER

Product Catalog

- Xem danh sách sản phẩm
- Xem chi tiết sản phẩm
- Chọn size, màu, chất liệu
- Admin/Staff quản lý sản phẩm cơ bản
- Kiểm tra tồn kho cơ bản

Design Editor

- Customer tạo thiết kế áo
- Thêm text
- Upload image
- Kéo thả, resize, rotate
- Lưu canvas JSON
- Lưu ảnh preview

AI Virtual Try-On

- Customer chọn design đã lưu
- Upload ảnh người dùng
- Gửi request Try-On
- AI xử lý async qua Kafka
- Hiển thị trạng thái: Pending, Processing, Success, Failed
- Xem ảnh Try-On preview

Trong demo, AI có thể dùng mock service trước. Tức là hệ thống vẫn có flow AI nhưng chưa cần model thật.

Order

- Customer tạo đơn hàng từ design đã lưu
- Hệ thống tạo order item
- Thanh toán giả lập
- Sau khi payment success, order chuyển sang paid
- Design bị khóa sau khi thanh toán

Production

- Staff xem đơn đã thanh toán
- Staff cập nhật trạng thái sản xuất
- Staff cập nhật trạng thái giao hàng cơ bản

Feedback

- Customer gửi feedback sau khi order completed
- Admin/Staff duyệt feedback
- Guest/Customer xem feedback đã approved

About Us

- Guest/Customer xem About Us
- Admin/Staff cập nhật nội dung About Us

2.2 Out of Scope

MVP chưa làm:

- Payment thật
 - Giao hàng thật
 - Refund
 - Voucher
 - Chat real-time
 - Multi-store
 - Multi-printing partner phức tạp
 - Dashboard analytics nâng cao
 - Permission chi tiết theo từng action
 - AI sinh thiết kế
 - Marketplace designer
 - Mobile app native
-

3. User Roles

3.1 Customer

Customer có thể:

- Đăng ký
 - Đăng nhập
 - Xem sản phẩm
 - Xem About Us
 - Xem feedback công khai
 - Thiết kế áo
 - Lưu design
 - Gửi yêu cầu Try-On
 - Xem Try-On result
 - Đặt hàng
 - Thanh toán giả lập
 - Theo dõi đơn
 - Gửi feedback
-

3.2 Staff

Staff là nhân sự vận hành demo.

Staff có thể:

- Xem đơn hàng
 - Xem file thiết kế
 - Cập nhật trạng thái sản xuất
 - Cập nhật trạng thái giao hàng
 - Kiểm tra tồn kho
 - Duyệt feedback cơ bản
-

3.3 Admin

Admin có thể:

- Quản lý user
- Quản lý sản phẩm
- Quản lý tồn kho
- Quản lý đơn hàng
- Quản lý About Us

- Quản lý feedback
 - Xem audit log cơ bản
-

4. Main Flow

- Guest visits website
 - ↓
 - View About Us / Product / Feedback
 - ↓
 - Register or Login
 - ↓
 - Browse Product
 - ↓
 - Select Product Variant
 - ↓
 - Open Design Editor
 - ↓
 - Create Design
 - ↓
 - Save Design
 - ↓
 - Request AI Try-On
 - ↓
 - AI Service processes async
 - ↓
 - View Try-On Result
 - ↓
 - Create Order
 - ↓
 - Mock Payment Success
 - ↓
 - Order Paid
 - ↓
 - Staff Updates Production
 - ↓
 - Staff Updates Shipping
 - ↓
 - Order Completed
 - ↓
 - Customer Submits Feedback
-

5. Business Rules

ID	Rule
BR-001	Customer phải đăng nhập để lưu thiết kế
BR-002	Design phải được lưu trước khi Try-On
BR-003	AI không được thay đổi design gốc
BR-004	Try-On result chỉ dùng để preview
BR-005	File dùng để in lấy từ Design, không lấy từ Try-On Result
BR-006	Không cho tạo order nếu product variant hết hàng
BR-007	Payment success thì order chuyển sang PAID

0
7

BR- Order PAID thì design bị LOCKED
0
0
8

BR- Staff chỉ cập nhật trạng thái production/shipping
0
0
9

BR- Customer chỉ feedback khi order COMPLETED
0
1
0

BR- Chỉ feedback APPROVED mới hiển thị công khai
0
1
1

BR- About Us chỉ hiển thị bản PUBLISHED
0
1
2

6. Service Design

6.1 Java Spring Boot Services

Service

Responsibility

Catalog Service	Product, variant, inventory
Design Service	Design, design layer, Try-On request
Order Service	Order, order item, order status
Feedback Service	Feedback
Content Service	About Us

6.2 C# .NET 8 Services

Service	Responsibility
Identity Service	Login, register, JWT, refresh token, role
Payment Service	Mock payment, payment status, payment event
Admin Service	Audit log, admin actions

6.3 AI Service

Service	Responsibility
---------	----------------

AI Try-On Service	Nhận Try-On request, xử lý mock/AI, trả result
-------------------	--

7. Kafka Scope

MVP chỉ cần các event quan trọng:

Event	Producer	Consumer	Purpose
DesignSaved	Design Service	AI Try-On Service	Biết design đã sẵn sàng
TryOnRequested	Design Service	AI Try-On Service	Gửi yêu cầu Try-On
TryOnCompleted	AI Try-On Service	Design Service	Cập nhật kết quả Try-On
OrderCreated	Order Service	Payment Service	Tạo payment request
PaymentSucceeded	Payment Service	Order Service	Cập nhật order PAID
OrderPaid	Order Service	Admin/Staff Notification	Báo có đơn cần xử lý
OrderCompleted	Order Service	Feedback Service	Cho phép customer feedback

Kafka Topics

Topic	Purpose
design.events	Design saved, Try-On requested
ai-tryon.events	Try-On completed/failed
order.events	Order created, order paid, order completed
payment.events	Payment succeeded/failed
feedback.events	Feedback submitted

8. Non-Functional Requirements

Requirement	Target
API response	< 2 seconds
Save design	< 3 seconds
Mock Try-On processing	< 10 seconds

Real Try-On target later	< 30 seconds
Checkout/payment mock	< 3 seconds
Security	JWT + RBAC
Logging	API log + Kafka event log
Database	PostgreSQL
Deployment demo	Docker Compose

9. Database Lite — 18 Tables

Table List

N	Table Name	Service Owner	Purpose
1	users	Identity Service	Lưu tài khoản
2	roles	Identity Service	Lưu role ADMIN, STAFF, CUSTOMER

3	user_roles	Identity Service	Gán role cho user
4	refresh_tokens	Identity Service	Quản lý refresh token
5	products	Catalog Service	Lưu sản phẩm áo
6	product_variants	Catalog Service	Lưu size, màu, chất liệu, SKU
7	product_images	Catalog Service	Lưu ảnh sản phẩm
8	product_inventory	Catalog Service	Lưu tồn kho
9	designs	Design Service	Lưu thiết kế
1	design_layers	Design Service	Lưu layer trong design
1	tryon_requests	AI/Design Service	Lưu yêu cầu Try-On
1	tryon_results	AI/Design Service	Lưu kết quả Try-On

1	orders	Order Service	Lưu đơn hàng
1	order_items	Order Service	Lưu sản phẩm trong đơn
1	payments	Payment Service	Lưu payment giả lập/thật sau này
1	order_status_history	Order Service	Lưu lịch sử trạng thái đơn
1	feedbacks	Feedback Service	Lưu feedback người dùng
1	about_us_contents	Content Service	Lưu nội dung About Us

10. Detailed Database Design

1. users

Lưu tài khoản người dùng.

Column	Type	Description
id	UUID	Primary key

email	VARCHA R	Email đăng nhập
password_ hash	TEXT	Mật khẩu đã hash
full_name	VARCHA R	Họ tên
phone	VARCHA R	Số điện thoại
avatar_url	TEXT	Ảnh đại diện
status	VARCHA R	ACTIVE, INACTIVE, BANNED
created_at	TIMESTA MP	Ngày tạo
updated_at	TIMESTA MP	Ngày cập nhật

2. roles

Lưu role hệ thống.

Column	Type	Description
--------	------	-------------

id	UUID	Primary key
code	VARCHAR	ADMIN, STAFF, CUSTOMER
name	VARCHAR	Tên hiển thị
description	TEXT	Mô tả
created_at	TIMESTAMP	Ngày tạo

Default data:

code	name
ADMIN	Quản trị viên
STAFF	Nhân viên
CUSTOMER	Khách hàng

3. user_roles

Gán role cho user.

Column	Type	Description
id	UUID	Primary key
user_id	UUID	User ID
role_id	UUID	Role ID
created_at	TIMESTAMP	Ngày tạo

Unique:

- user_id + role_id

4. refresh_tokens

Quản lý refresh token.

Column	Type	Description
id	UUID	Primary key
user_id	UUID	User ID

token_ has h	TEXT	Refresh token hash
--------------------	------	-----------------------

expires _at	TIMESTA MP	Hết hạn
----------------	---------------	---------

revoke d_a t	TIMESTA MP	Thu hồi
--------------------	---------------	---------

created _at	TIMESTA MP	Ngày tạo
----------------	---------------	----------

5. products

Lưu sản phẩm áo.

Colum n	Type	Description
id	UUID	Primary key
name	VARCHA R	Tên sản phẩm
descrip tion	TEXT	Mô tả

base_ price	DECIMAL	Giá cơ bản
status	VARCHAR	DRAFT, ACTIVE, INACTIVE
created_ by	UUID	Admin/Staff tạo
created_ at	TIMESTAMP	Ngày tạo
updated_ at	TIMESTAMP	Ngày cập nhật

6. product_variants

Lưu biến thể sản phẩm.

Column	Type	Description
id	UUID	Primary key
product_id	UUID	Product ID
sku	VARCHAR	Mã SKU

size	VARCHA R	S, M, L, XL
color	VARCHA R	White, Black
material	VARCHA R	Cotton
price_adjust ment	DECIMAL	Chênh lệch giá
status	VARCHA R	ACTIVE, INACTIVE
created_at	TIMESTA MP	Ngày tạo

7. product_images

Lưu ảnh sản phẩm.

Column	Type	Descript ion
id	UUID	Primary key
product_i d	UUID	Product ID

image_url	TEXT	URL ảnh
-----------	------	---------

is_thumbnail	BOOLEAN	Ảnh đại diện
--------------	---------	--------------

sort_order	INT	Thứ tự
------------	-----	--------

created_at	TIMESTAMP	Ngày tạo
------------	-----------	----------

8. product_inventory

Lưu tồn kho theo variant.

Column	Type	Description
id	UUID	Primary key
product_variant_id	UUID	Variant ID
available_quantity	INT	Số lượng có thể bán
reserved_quantity	INT	Số lượng đang giữ

sold_quantity	INT	Số lượng đã bán
updated_at	TIMESTAMP	Ngày cập nhật

Business rule:

- available_quantity - reserved_quantity > 0

9. designs

Lưu thiết kế của Customer.

Column	Type	Description
id	UUID	Primary key
customer_id	UUID	Customer ID
product_id	UUID	Product ID
product_variant_id	UUID	Variant ID
name	VARCHAR	Tên design
canvas_json	JSONB	Dữ liệu Fabric.js

preview_image_url	TEXT	Ảnh preview
print_file_url	TEXT	File dùng để in
status	VARCHAR	DRAFT, SAVED, LOCKED
created_at	TIMESTAMP	Ngày tạo
updated_at	TIMESTAMP	Ngày cập nhật

10. design_layers

Lưu layer trong design.

Column	Type	Description
id	UUID	Primary key
design_id	UUID	Design ID
layer_type	VARCHAR	TEXT, IMAGE, ICON

content	TEXT	Nội dung text hoặc URL asset
position_x	DECIMAL	Vị trí X
position_y	DECIMAL	Vị trí Y
width	DECIMAL	Chiều rộng
height	DECIMAL	Chiều cao
rotation	DECIMAL	Góc xoay
color	VARCHAR	Màu
z_index	INT	Thứ tự layer

Ghi chú: vẫn lưu `canvas_json` trong bảng `designs`. `design_layers` dùng để dễ query, debug và học database relationship.

11. tryon_requests

Lưu yêu cầu Try-On.

Column	Type	Description
id	UUID	Primary key
customer_id	UUID	Customer ID
design_id	UUID	Design ID
user_photo_url	TEXT	Ảnh người dùng upload
height_cm	DECIMAL	Chiều cao
weight_kg	DECIMAL	Cân nặng
status	VARCHAR	PENDING, PROCESSING, SUCCEEDED, FAILED
error_message	TEXT	Lỗi nếu có
requested_at	TIMESTAMP	Ngày yêu cầu
completed_at	TIMESTAMP	Ngày hoàn tất

12. tryon_results

Lưu kết quả Try-On.

Column	Type	Description
id	UUID	Primary key
tryon_request_id	UUID	Request ID
design_id	UUID	Design ID
result_image_url	TEXT	Ảnh Try-On preview
processing_time_ms	INT	Thời gian xử lý
created_at	TIMESTAMP	Ngày tạo

Business rule:

- tryon_results.result_image_url không được dùng làm file in

13. orders

Lưu đơn hàng.

Column	Type	Description
--------	------	-------------

id	UUID	Primary key
order_code	VARCHAR	Mã đơn
customer_id	UUID	Customer ID
total_amount	DECIMAL	Tổng tiền
payment_status	VARCHAR	PENDING, PAID, FAILED
order_status	VARCHAR	PENDING_PAYMENT, PAID, IN_PRODUCTION, SHIPPING, COMPLETED, CANCELLED
receiver_name	VARCHAR	Người nhận
receiver_phone	VARCHAR	SĐT
shipping_address	TEXT	Địa chỉ giao hàng
created_at	TIMESTAMP	Ngày tạo
updated_at	TIMESTAMP	Ngày cập nhật

14. order_items

Lưu sản phẩm trong đơn.

Column	Type	Description
id	UUID	Primary key
order_id	UUID	Order ID
product_id	UUID	Product ID
product_variant_id	UUID	Variant ID
design_id	UUID	Design ID
product_name_snapshot	VARCHAR	Tên sản phẩm lúc mua
variant_snapshot	JSONB	Size, màu, chất liệu lúc mua
quantity	INT	Số lượng
unit_price	DECIMAL	Đơn giá
total_price	DECIMAL	Tổng tiền

15. payments

Lưu thông tin thanh toán.

Column	Type	Description
id	UUID	Primary key
order_id	UUID	Order ID
customer_id	UUID	Customer ID
amount	DECIMAL	Số tiền
payment_method	VARCHAR	MOCK, VNPAY, MOMO
payment_status	VARCHAR	PENDING, PAID, FAILED
transaction_code	VARCHAR	Mã giao dịch
paid_at	TIMESTAMP	Thời điểm thanh toán
created_at	TIMESTAMP	Ngày tạo

Trong demo, payment có thể dùng **MOCK**.

16. order_status_history

Lưu lịch sử trạng thái đơn.

Column	Type	Description
id	UUID	Primary key
order_id	UUID	Order ID
from_status	VARCHAR	Trạng thái cũ
to_statuses	VARCHAR	Trạng thái mới
changing_user	UUID	User cập nhật
note	TEXT	Ghi chú
created_at	TIMESTAMP	Ngày tạo

17. feedbacks

Lưu feedback của Customer.

Column	Type	Description
id	UUID	Primary key
customer_id	UUID	Customer ID
order_id	UUID	Order ID
product_id	UUID	Product ID
rating	INT	1-5
comment	TEXT	Nội dung
image_url	TEXT	Ảnh optional
status	VARCHAR	PENDING, APPROVED, HIDDEN, REJECTED
reviewed_by	UUID	Admin/Staff duyệt
created_at	TIMESTAMP	Ngày tạo

updated_at	TIMESTAMP	Ngày cập nhật
------------	-----------	---------------

18. about_us_contents

Lưu nội dung About Us.

Column	Type	Description
id	UUID	Primary key
section_key	VARCHAR	INTRODUCTION, MISSION, HOW_IT_WORKS
title	VARCHAR	Tiêu đề
content	TEXT	Nội dung
image_url	TEXT	Ảnh minh họa
status	VARCHAR	DRAFT, PUBLISHED
updated_by	UUID	Admin/Staff cập nhật

updated _at	TIMESTA MP	Ngày cập nhật
----------------	---------------	---------------

11. Source of Truth

Data	Source of Truth
User, Role	Identity Service
Product, Variant, Inventory	Catalog Service
Design	Design Service
Try-On Request/Result	AI Try-On Service
Order	Order Service
Payment	Payment Service
Feedback	Feedback Service

12. PRD To Database Mapping

PRD Feature	Tables
Register/Log in	users, roles, user_roles, refresh_tokens
Product Catalog	products, product_variants, product_images
Inventory Check	product_inventory
Design Editor	designs, design_layers
AI Try-On	tryon_requests, tryon_results
Create Order	orders, order_items
Payment Mock	payments
Track Order	orders, order_status_history

Feedback

feedbacks

About Us

about_us_contents

13. Learning Value

Bản giới hạn này vẫn giúp học được các phần quan trọng:

Database

- One-to-many relationship
- Many-to-many relationship
- Snapshot data
- Status history
- JSONB
- Inventory check
- RBAC cơ bản

Backend

- Authentication
- Authorization
- CRUD
- Transaction
- Validation
- Error handling
- Service ownership

Kafka

- Event producer
- Event consumer
- Async processing
- Event retry
- Event-driven order/payment flow

System Design

- Source of Truth
 - Service boundary
 - Async AI processing
 - Payment event
 - Order lifecycle
 - Lock design after payment
-

14. Final MVP Learning Scope

MVP chỉ cần chứng minh 5 flow chính:

Flow 1: Authentication

- Register
- → Login
- → JWT
- → Role check

Flow 2: Design

- Select Product
- → Create Design
- → Save Design
- → DesignSaved Event

Flow 3: AI Try-On

- Request Try-On
- → Kafka TryOnRequested
- → AI Service Processing
- → Kafka TryOnCompleted
- → Show Result

Flow 4: Order & Payment

- Create Order

- → OrderCreated Event
- → Mock Payment
- → PaymentSucceeded Event
- → Order PAID
- → Lock Design

Flow 5: Production & Feedback

- Staff Update Order Status
 - → Order Completed
 - → Customer Submit Feedback
 - → Admin/Staff Approve Feedback
-

15. Final Decision

Thay vì dùng 38 bảng, MVP Learning Version dùng **18 bảng**.

Bản này đủ để demo:

- Có login/register
- Có 3 role
- Có product catalog
- Có design editor
- Có AI Try-On async
- Có Kafka
- Có order/payment mock
- Có staff xử lý đơn
- Có feedback
- Có About Us
-

SPEC-Common

02-DB Design Spec

02. DATABASE DESIGN DOC — MVP LEARNING VERSION

Balanced Java + C# Backend Scope

Project: AI T-Shirt Customization & Virtual Try-On Platform

Version: 2.0

Based on: PRD Lite v3.0

Database: PostgreSQL

Tables: 18 tables

Backend: Java Spring Boot + C# .NET 8

Messaging: Kafka

MVP Roles: ADMIN, STAFF, CUSTOMER

1. Document Purpose

Tài liệu này thiết kế database và chia ownership backend cho MVP theo hướng:

- Giữ database nhỏ gọn, chỉ **18 bảng**
 - Đủ để demo end-to-end
 - Đủ để học system design, Kafka, transaction, status flow
 - Java có phần core business phức tạp hơn
 - C# có phần thực tế, vừa sức fresher cứng
 - Hai bên có trách nhiệm rõ, không đè việc lên một bên
-

2. Backend Level Target

2.1 Java Backend Target

Java Spring Boot được dùng cho các phần core domain chính.

Mục tiêu level:

Junior cứng → Middle mềm

Java sẽ học và xử lý:

- Product domain
- Design domain
- Order lifecycle
- Inventory check
- Kafka producer/consumer
- Transaction boundary
- Status flow
- Business rule validation
- Event-driven integration
- Clean Architecture mức vừa phải

Java không chỉ CRUD, mà phải xử lý logic nghiệp vụ chính.

2.2 C# Backend Target

C# .NET 8 được dùng cho các phần vận hành và integration vừa sức.

Mục tiêu level:

Fresher cứng-Junior thấp

C# sẽ học và xử lý:

- Authentication
- JWT
- Refresh token
- RBAC 3 role
- Mock payment
- Payment status
- Generate invoice PDF
- Tạo file hóa đơn
- Xử lý print package cơ bản
- Staff production actions
- Admin/Staff operation API
- Consume/produce Kafka event đơn giản

C# không cần gánh domain quá phức tạp, nhưng vẫn có task thực tế để lên tay.

3. MVP Role Scope

MVP chỉ dùng 3 role:

ADMIN

STAFF

CUSTOMER

3.1 ADMIN

Admin có thể:

- Quản lý user
- Quản lý product
- Quản lý inventory
- Quản lý About Us
- Quản lý feedback
- Xem order
- Theo dõi payment
- Theo dõi production status

3.2 STAFF

Staff có thể:

- Xem order đã thanh toán
- Xem file thiết kế dùng để in
- Cập nhật trạng thái sản xuất
- Cập nhật trạng thái giao hàng
- Tải hóa đơn PDF
- Duyệt feedback cơ bản

3.3 CUSTOMER

Customer có thể:

- Đăng ký
 - Đăng nhập
 - Xem sản phẩm
 - Thiết kế áo
 - Try-On
 - Tạo order
 - Thanh toán mock
 - Theo dõi order
 - Gửi feedback
 - Xem About Us
-

4. Database Strategy

MVP dùng một PostgreSQL database:

ai_tshirt_mvp_db

Nhưng chia schema logic theo service để học service boundary:

identity
catalog
design
ai_tryon
ordering
payment
feedback
content

Không thêm bảng mới ngoài 18 bảng.

5. Service Ownership Overview

Area	Service	Tech	Level Target
Identity	Identity Service	C# .NET 8	Fresher cứng
Payment & Invoice	Payment Service	C# .NET 8	Fresher cứng
Staff Production Actions	Operation Service	C# .NET 8	Fresher cứng
Product Catalog	Catalog Service	Java Spring Boot	Junior cứng
Design Editor Backend	Design Service	Java Spring Boot	Junior cứng
AI Try-On Orchestration	Try-On Service	Java Spring Boot / Python mock	Junior cứng
Order Lifecycle	Order Service	Java Spring Boot	Junior cứng / Middle mềm

Feedback	Feedback Service	Java Spring Boot	Junior
About Us	Content Service	Java Spring Boot hoặc C#	Fresher/Junior

6. Balanced Responsibility Split

6.1 Java Responsibility

Java owns core business domain.

Module	Tables	Responsibility
Catalog	products, product_variants, product_images, product_inventory	Product, variant, image, stock
Design	designs, design_layers	Save design, validate design, lock design
AI Try-On Orchestration	tryon_requests, tryon_results	Request Try-On, update result
Order	orders, order_items, order_status_history	Create order, update order status, track lifecycle
Feedback	feedbacks	Submit feedback, approve feedback
Content	about_us_contents	About Us CRUD nếu team Java nhận

Java nên tập trung vào:

- Business rules
 - Transaction
 - Kafka event
 - Domain logic
 - Order state machine
 - Integration với C#
-

6.2 C# Responsibility

C# owns platform support and operation features.

Module	Tables	Responsibility
Identity	users, roles, user_roles, refresh_tokens	Register, login, JWT, refresh token, RBAC
Payment	payments	Mock payment, payment status, PaymentSucceeded event
Invoice PDF	payments + order data from Java API	Generate invoice PDF
Staff Operation	orders via Java API, order_status_history via Java API	Staff update production/shipping
Print Package	designs + orders via Java API	Tạo package thông tin in
Admin Operation	users, roles, product/order API call	Admin/Staff basic management

C# nên tập trung vào:

- API chuẩn
- Security
- JWT
- PDF generation
- Payment mock
- Kafka event đơn giản
- Gọi API Java đúng boundary
- Không sửa trực tiếp bảng của Java nếu đang theo service boundary

7. Important Boundary Rule

Dù MVP dùng chung database, vẫn nên tập suy nghĩ theo service boundary.

Rule

Service nào owns table thì service đó mới được update table đó.

Ví dụ:

- C# Payment Service được update bằng **payments**
- Java Order Service được update bằng **orders**

- C# không update trực tiếp **orders**
- C# gửi Kafka event **PaymentSucceeded**
- Java Order Service consume event rồi update **orders**

8. Full Table List — 18 Tables

No.	Table Name	Owner	Tech	Purpose
1	users	Identity Service	C#	Lưu tài khoản
2	roles	Identity Service	C#	Lưu role
3	user_roles	Identity Service	C#	Gán role cho user
4	refresh_tokens	Identity Service	C#	Quản lý refresh token
5	products	Catalog Service	Java	Lưu sản phẩm
6	product_variants	Catalog Service	Java	Lưu size, màu, chất liệu
7	product_images	Catalog Service	Java	Lưu ảnh sản phẩm
8	product_inventory	Catalog Service	Java	Quản lý tồn kho
9	designs	Design Service	Java	Lưu thiết kế
10	design_layers	Design Service	Java	Lưu layer thiết kế
11	tryon_requests	Try-On Service	Java	Lưu request Try-On
12	tryon_results	Try-On Service	Java	Lưu kết quả Try-On
13	orders	Order Service	Java	Lưu đơn hàng
14	order_items	Order Service	Java	Lưu sản phẩm trong đơn
15	payments	Payment Service	C#	Lưu thanh toán, invoice info
16	order_status_history	Order Service	Java	Lưu lịch sử trạng thái đơn
17	feedbacks	Feedback Service	Java	Lưu feedback
18	about_us_contents	Content Service	Java/C#	Lưu About Us

9. Recommended Ownership Ratio

Không cần chia đều số bảng tuyệt đối. Chia đều theo độ khó và learning value.

Java

Java có nhiều bảng hơn vì chịu core domain:

products
product_variants
product_images
product_inventory
designs
design_layers
tryon_requests
tryon_results
orders
order_items
order_status_history
feedbacks

Java học được:

- Domain modeling
- Transaction
- Order lifecycle
- Kafka consumer
- Kafka producer
- Validation
- State transition

C#

C# có ít bảng hơn nhưng có task thực tế:

users
roles
user_roles
refresh_tokens
payments

C# học được:

- Authentication
- Authorization

- JWT
- Payment mock
- Invoice PDF
- Kafka producer
- API integration
- Staff operation flow

Như vậy là cân bằng theo level:

Java = core complexity

C# = platform + operation + invoice/payment

10. Detailed Database Design

10.1 users

Owner

C# Identity Service

Purpose

Lưu tài khoản người dùng.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
email	VARCHAR(255)	Yes	Email đăng nhập
password_hash	TEXT	Yes	Mật khẩu đã hash
full_name	VARCHAR(255)	Yes	Họ tên
phone	VARCHAR(20)	No	Số điện thoại
avatar_url	TEXT	No	Ảnh đại diện

status	VARCHAR(30)	Yes	ACTIVE, INACTIVE, BANNED
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Rules

- Email unique
- Password phải hash
- User phải có ít nhất một role

10.2 roles

Owner

C# Identity Service

Purpose

Lưu role hệ thống.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
code	VARCHAR(50)	Yes	ADMIN, STAFF, CUSTOMER
name	VARCHAR(100)	Yes	Tên role
description	TEXT	No	Mô tả
created_at	TIMESTAMP	Yes	Ngày tạo

Default Data

code	name
ADMIN	Quản trị viên

STAFF	Nhân viên
CUSTOMER	Khách hàng

10.3 user_roles

Owner

C# Identity Service

Purpose

Gán role cho user.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
user_id	UUID	Yes	User ID
role_id	UUID	Yes	Role ID
created_at	TIMESTAMP	Yes	Ngày tạo

Rules

- Unique `user_id + role_id`
 - Admin mới được thay đổi role
-

10.4 refresh_tokens

Owner

C# Identity Service

Purpose

Quản lý refresh token.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
user_id	UUID	Yes	User ID
token_hash	TEXT	Yes	Refresh token hash
expires_at	TIMESTAMP	Yes	Hết hạn
revoked_at	TIMESTAMP	No	Thu hồi
created_at	TIMESTAMP	Yes	Ngày tạo

10.5 products

Owner

Java Catalog Service

Purpose

Lưu sản phẩm áo.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
name	VARCHAR(255)	Yes	Tên sản phẩm
description	TEXT	No	Mô tả
base_price	DECIMAL(18,2)	Yes	Giá cơ bản
status	VARCHAR(30)	Yes	DRAFT, ACTIVE, INACTIVE

created_by	UUID	No	Admin/Staff tạo
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Java Learning Value

- Product CRUD
 - Validation
 - Search/filter cơ bản
 - Status management
-

10.6 product_variants

Owner

Java Catalog Service

Purpose

Lưu biến thể sản phẩm.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
product_id	UUID	Yes	Product ID
sku	VARCHAR(100)	Yes	Mã SKU
size	VARCHAR(50)	Yes	Size áo
color	VARCHAR(100)	Yes	Màu áo
material	VARCHAR(100)	Yes	Chất liệu
price_adjustment	DECIMAL(18,2)	Yes	Chênh lệch giá

status	VARCHAR(30)	Yes	ACTIVE, INACTIVE
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Rules

- SKU unique
 - Variant inactive thì không cho order
-

10.7 product_images

Owner

Java Catalog Service

Purpose

Lưu ảnh sản phẩm.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
product_id	UUID	Yes	Product ID
image_url	TEXT	Yes	URL ảnh
is_thumbnail	BOOLEAN	Yes	Ảnh đại diện
sort_order	INT	Yes	Thứ tự hiển thị
created_at	TIMESTAMP	Yes	Ngày tạo

10.8 product_inventory

Owner

Java Catalog Service

Purpose

Quản lý tồn kho.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
product_variant_id	UUID	Yes	Variant ID
available_quantity	INT	Yes	Số lượng có thể bán
reserved_quantity	INT	Yes	Số lượng đang giữ
sold_quantity	INT	Yes	Số lượng đã bán
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Rules

Cho phép order khi:

`available_quantity - reserved_quantity >= order_quantity`

Java Learning Value

- Inventory validation
- Transaction
- Race condition awareness
- Stock reservation basic

10.9 designs

Owner

Purpose

Lưu thiết kế gốc của Customer.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
customer_id	UUID	Yes	Customer ID
product_id	UUID	Yes	Product ID
product_variant_id	UUID	Yes	Variant ID
name	VARCHAR(255)	Yes	Tên design
canvas_json	JSONB	Yes	Dữ liệu Fabric.js
preview_image_url	TEXT	No	Ảnh preview
print_file_url	TEXT	No	File dùng để in
status	VARCHAR(30)	Yes	DRAFT, SAVED, LOCKED
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Rules

- Design phải **SAVED** trước khi Try-On
- Design chuyển **LOCKED** sau khi order paid
- AI không được sửa **canvas_json**
- File in lấy từ **print_file_url**

10.10 design_layers

Owner

Purpose

Lưu layer trong thiết kế.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
design_id	UUID	Yes	Design ID
layer_type	VARCHAR(50)	Yes	TEXT, IMAGE, ICON
content	TEXT	No	Text hoặc URL
position_x	DECIMAL(10,2)	Yes	Vị trí X
position_y	DECIMAL(10,2)	Yes	Vị trí Y
width	DECIMAL(10,2)	Yes	Chiều rộng
height	DECIMAL(10,2)	Yes	Chiều cao
rotation	DECIMAL(10,2)	Yes	Góc xoay
color	VARCHAR(50)	No	Màu
z_index	INT	Yes	Thứ tự layer

10.11 tryon_requests

Owner

Java Try-On Orchestration Service

Purpose

Lưu yêu cầu AI Virtual Try-On.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
customer_id	UUID	Yes	Customer ID
design_id	UUID	Yes	Design ID
user_photo_url	TEXT	Yes	Ảnh user upload
height_cm	DECIMAL(5,2)	No	Chiều cao
weight_kg	DECIMAL(5,2)	No	Cân nặng
status	VARCHAR(30)	Yes	PENDING, PROCESSING, SUCCEEDED, FAILED
error_message	TEXT	No	Lỗi nếu có
requested_at	TIMESTAMP	Yes	Ngày request
completed_at	TIMESTAMP	No	Ngày hoàn tất

Kafka

Khi tạo request:

Produce event: TryOnRequested

Topic: design.events

10.12 tryon_results

Owner

Java Try-On Orchestration Service

Purpose

Lưu kết quả Try-On.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
tryon_request_id	UUID	Yes	Request ID
design_id	UUID	Yes	Design ID
result_image_url	TEXT	Yes	Ảnh Try-On preview
processing_time_ms	INT	No	Thời gian xử lý
created_at	TIMESTAMP	Yes	Ngày tạo

Rules

- Result chỉ dùng preview
 - Không dùng result image làm file in
-

10.13 orders

Owner

Java Order Service

Purpose

Lưu đơn hàng.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
order_code	VARCHAR(100)	Yes	Mã đơn
customer_id	UUID	Yes	Customer ID

total_amount	DECIMAL(18,2)	Yes	Tổng tiền
payment_status	VARCHAR(30)	Yes	PENDING, PAID, FAILED
order_status	VARCHAR(50)	Yes	PENDING_PAYMENT, PAID, IN_PRODUCTION, SHIPPING, COMPLETED, CANCELLED
receiver_name	VARCHAR(255)	Yes	Người nhận
receiver_phone	VARCHAR(20)	Yes	SĐT
shipping_addresses	TEXT	Yes	Địa chỉ giao hàng
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Java Learning Value

Đây là bảng quan trọng nhất để học backend domain.

Java cần xử lý:

- Create order transaction
- Validate product variant
- Check inventory
- Calculate total amount
- Create order items
- Publish OrderCreated event
- Consume PaymentSucceeded event
- Update order status
- Insert order status history
- Lock design after payment

10.14 order_items

Owner

Java Order Service

Purpose

Lưu sản phẩm trong đơn hàng.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
order_id	UUID	Yes	Order ID
product_id	UUID	Yes	Product ID
product_variant_id	UUID	Yes	Variant ID
design_id	UUID	Yes	Design ID
product_name_snapshot	VARCHAR(255)	Yes	Tên sản phẩm lúc mua
variant_snapshot	JSONB	Yes	Size, màu, chất liệu lúc mua
quantity	INT	Yes	Số lượng
unit_price	DECIMAL(18,2)	Yes	Đơn giá
total_price	DECIMAL(18,2)	Yes	Tổng tiền

Rules

- Snapshot data không đổi sau khi order tạo
- **design_id** phải là design của customer
- Không dùng Try-On result làm file in

10.15 payments

Owner

C# Payment Service

Purpose

Lưu thông tin thanh toán mock và thông tin invoice cơ bản.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
order_id	UUID	Yes	Order ID
customer_id	UUID	Yes	Customer ID
amount	DECIMAL(18,2)	Yes	Số tiền
payment_method	VARCHAR(50)	Yes	MOCK, VNPAY, MOMO
payment_status	VARCHAR(30)	Yes	PENDING, PAID, FAILED
transaction_code	VARCHAR(100)	No	Mã giao dịch
invoice_number	VARCHAR(100)	No	Mã hóa đơn
invoice_pdf_url	TEXT	No	URL file PDF hóa đơn
paid_at	TIMESTAMP	No	Thời điểm thanh toán
created_at	TIMESTAMP	Yes	Ngày tạo

Notes

Vẫn giữ 18 bảng. Chỉ bổ sung thêm 2 cột để C# có phần xử lý PDF hóa đơn:

invoice_number
invoice_pdf_url

C# Learning Value

C# xử lý:

- Create payment record
- Mock payment success/fail
- Validate amount
- Generate transaction code
- Generate invoice number
- Generate invoice PDF
- Store invoice PDF URL
- Produce PaymentSucceeded event

Kafka

Khi payment thành công:

Produce event: PaymentSucceeded

Topic: payment.events

10.16 order_status_history

Owner

Java Order Service

Purpose

Lưu lịch sử trạng thái đơn hàng.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
order_id	UUID	Yes	Order ID
from_status	VARCHAR(50)	No	Trạng thái cũ
to_status	VARCHAR(50)	Yes	Trạng thái mới
changed_by	UUID	No	User cập nhật
note	TEXT	No	Ghi chú
created_at	TIMESTAMP	Yes	Ngày tạo

Staff Operation Rule

Staff có thể yêu cầu cập nhật order status qua C# Operation API, nhưng update thật nên đi qua Java Order API.

Ví dụ:

C# Staff API
→ call Java Order API
→ Java update orders
→ Java insert order_status_history

10.17 feedbacks

Owner

Java Feedback Service

Purpose

Lưu feedback của customer.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
customer_id	UUID	Yes	Customer ID
order_id	UUID	Yes	Order ID
product_id	UUID	Yes	Product ID
rating	INT	Yes	1-5
comment	TEXT	No	Nội dung
image_url	TEXT	No	Ảnh feedback
status	VARCHAR(30)	Yes	PENDING, APPROVED, HIDDEN, REJECTED
reviewed_by	UUID	No	Admin/Staff duyệt
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Rules

- Customer chỉ feedback khi order **COMPLETED**
 - Feedback default là **PENDING**
 - Chỉ feedback **APPROVED** mới public
-

10.18 about_us_contents

Owner

Content Service

Recommended Owner

Có thể để C# hoặc Java. Để chia việc đều hơn, nên giao cho C# nếu Java đã nhiều việc.

Recommended for balance: C# Content/Admin Service

Purpose

Lưu nội dung About Us.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
section_key	VARCHAR(100)	Yes	INTRODUCTION, MISSION, HOW_IT_WORKS
title	VARCHAR(255)	Yes	Tiêu đề
content	TEXT	Yes	Nội dung
image_url	TEXT	No	Ảnh
status	VARCHAR(30)	Yes	DRAFT, PUBLISHED

updated_by	UUID	No	Admin/Staff cập nhật
updated_at	TIMESTAMP	Yes	Ngày cập nhật

11. Final Ownership By Team

Java Team Scope

Java làm các service sau:

Catalog Service
Design Service
Try-On Orchestration Service
Order Service
Feedback Service

Java owns 12 tables:

products
product_variants
product_images
product_inventory
designs
design_layers
tryon_requests
tryon_results
orders
order_items
order_status_history
feedbacks

Java main tasks:

- Product CRUD
- Variant CRUD
- Inventory check
- Save design
- Save design layers
- Create Try-On request
- Consume TryOnCompleted
- Create order

- Create order item snapshot
 - Consume PaymentSucceeded
 - Lock design
 - Update order status
 - Feedback submit/approve
-

C# Team Scope

C# làm các service sau:

Identity Service

Payment & Invoice Service

Content/Admin Service

Operation Gateway for Staff

C# owns 6 tables:

users

roles

user_roles

refresh_tokens

payments

about_us_contents

C# main tasks:

- Register
 - Login
 - JWT
 - Refresh token
 - RBAC 3 role
 - Mock payment
 - Generate transaction code
 - Generate invoice PDF
 - Store invoice URL
 - About Us management
 - Staff operation API
 - Consume OrderCreated event
 - Produce PaymentSucceeded event
-

12. Kafka Event Flow

12.1 Try-On Flow

Java Design Service

- create tryon_requests
- publish TryOnRequested
- AI Worker / Mock AI consumes
- generate result image
- publish TryOnCompleted
- Java Try-On Service updates tryon_results

12.2 Order Payment Flow

Java Order Service

- create orders + order_items
- publish OrderCreated
- C# Payment Service consumes
- create payments
- mock payment success
- generate invoice PDF
- publish PaymentSucceeded
- Java Order Service consumes
- update order PAID
- lock design
- insert order_status_history

12.3 Staff Production Flow

Staff UI

- C# Operation API
- validate STAFF role
- call Java Order API
- Java updates order_status
- Java inserts order_status_history

12.4 Feedback Flow

Java Order Service

- publish OrderCompleted
 - Customer can submit feedback
 - Java Feedback Service stores feedback
 - Staff/Admin approves feedback
-

13. API Boundary Between Java And C#

13.1 C# Calls Java

C# cần gọi Java trong các case:

Use Case	C# Action	Java API
Generate invoice PDF	Get order detail	GET /orders/{id}
Generate invoice PDF	Get order items	GET /orders/{id}/items
Generate invoice PDF	Get design print file	GET /designs/{id}/print-file
Staff update production	Request status update	PATCH /orders/{id}/status
Staff view order	Get order detail	GET /orders/{id}

13.2 Java Calls C#

Java cần gọi C# trong các case:

Use Case	Java Action	C# API
Validate user	Check user exists	GET /users/{id}
Check role optional	Validate role	GET /users/{id}/roles
Create payment manually	Request payment	POST /payments

Trong MVP, ưu tiên Kafka cho payment flow thay vì Java call trực tiếp C#.

14. Main End-To-End Flow

Customer Login

→ C# Identity Service

Customer View Product

→ Java Catalog Service

Customer Create Design

→ Java Design Service

Customer Request Try-On

→ Java Try-On Service

→ Kafka TryOnRequested

→ AI Worker

→ Kafka TryOnCompleted

→ Java saves Try-On Result

Customer Create Order

→ Java Order Service

→ Java publishes OrderCreated

C# Payment Service consumes OrderCreated

→ Creates payment

→ Mock payment success

→ Generates invoice PDF

→ Publishes PaymentSucceeded

Java Order Service consumes PaymentSucceeded

→ Updates order PAID

→ Locks design

→ Inserts order status history

Staff processes order

→ C# Operation API validates STAFF

→ Calls Java Order API

→ Java updates status IN_PRODUCTION / SHIPPING / COMPLETED

Customer submits feedback

→ Java Feedback Service

Guest views About Us

→ C# Content Service

15. Learning Outcome

Java Learning Outcome

Sau khi làm xong, Java dev hiểu được:

- Spring Boot REST API
- Entity relationship

- Transaction
- Validation
- Inventory check
- Order creation
- Snapshot data
- Status history
- Kafka producer
- Kafka consumer
- Event-driven order/payment flow
- Design lock rule
- Domain service

C# Learning Outcome

Sau khi làm xong, C# dev hiểu được:

- ASP.NET Core Web API
 - JWT authentication
 - Refresh token
 - RBAC
 - Password hashing
 - Mock payment flow
 - PDF invoice generation
 - File storage URL
 - Kafka producer/consumer cơ bản
 - Calling Java service API
 - Staff operation gateway
-

16. Why This Split Is Balanced

Không nên cân bằng bằng số lượng bảng, mà cân bằng bằng độ khó và giá trị học.

Java nhiều bảng hơn vì:

- Java là core business
- Order/design/catalog cần nhiều business rule
- Java mục tiêu lên junior cứng/middle mềm

C# ít bảng hơn nhưng task thực tế hơn vì:

- Auth là nền tảng quan trọng
- Payment là flow nhạy cảm
- Invoice PDF là nghiệp vụ thật

- Operation API giúp học integration
- C# mục tiêu fresher cứng, không bị quá tải

Kết luận:

Java = Core domain + Kafka + transaction

C# = Identity + Payment + Invoice PDF + Operation integration

Đây là cách chia phù hợp nhất cho MVP học tập nhưng vẫn có chất lượng hệ thống.

17. Final Decision

Database vẫn giữ **18 bảng**.

Có điều chỉnh nhỏ:

- Không thêm bảng mới
- Thêm optional columns vào **payments**:
 - **invoice_number**
 - **invoice_pdf_url**

Lý do:

- Để C# có phần xử lý PDF hóa đơn thực tế
- Không làm database phình to
- Vẫn giữ đúng MVP Learning Scope

Final service split:

Sid	Main Role
e	
Java	Core product, design, AI orchestration, order, feedback
a	
C#	Auth, payment, invoice PDF, About Us/Admin content, staff operation gateway

Tài liệu này là bản database + ownership chính thức cho MVP Learning Version.

03-BackEnd Spec API

AC03. BACKEND API SPECIFICATION

AI T-Shirt Customization & Virtual Try-On Platform

Version: 2.0
Based on: PRD Lite + Database Design 18 Tables
Backend: Java Spring Boot + C# .NET 8
Payment: Real Payment Provider Ready — PayOS / SePay
Messaging: Kafka
MVP Roles: ADMIN, STAFF, CUSTOMER

1. Document Purpose

Tài liệu này mô tả toàn bộ API backend cho MVP.

API doc này trả lời 5 câu hỏi chính cho từng API:

- 1. API này dùng để làm gì?
 - 2. Ai được gọi API này?
 - 3. Frontend gọi API này ở màn hình nào?
 - 4. Backend xử lý logic gì?
 - 5. API trả về gì khi thành công hoặc thất bại?
-

2. API Ownership

2.1 C# .NET 8 Owns

C# phụ trách các API thiên về hệ thống, xác thực, thanh toán và vận hành.

Domain	C# Responsibility
Authentication	Register, Login, Refresh Token, Logout
Authorization	JWT, RBAC 3 role
Payment	PayOS/SePay payment, webhook, payment status

Invoice	Generate invoice PDF
About Us	Content management
Staff Operation Gateway	Validate STAFF/ADMIN rồi gọi Java API nếu cần

2.2 Java Spring Boot Owns

Java phụ trách core business domain.

Domain	Java Responsibility
Catalog	Product, variant, image, inventory
Design	Save design, design layer, lock design
Try-On	Try-On request, Try-On result
Order	Create order, order item, order status
Feedback	Submit and moderate feedback

3. Standard API Response

3.1 Success Response Format

```
{
  "success": true,
  "message": "Success",
  "data": {},
  "errors": null,
  "meta": {
    "requestId": "req_123456",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

3.2 Error Response Format

```
{
```

```
"success": false,
"message": "Validation failed",
"data": null,
"errors": [
  {
    "field": "email",
    "code": "EMAIL_REQUIRED",
    "message": "Email is required"
  }
],
"meta": {
  "requestId": "req_123456",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

4. HTTP Status Code Standard

Statu s	Meaning	Use Case
200	OK	Lấy dữ liệu, cập nhật thành công
201	Created	Tạo dữ liệu mới thành công
202	Accepted	Request async đã được nhận
204	No Content	Xóa thành công
400	Bad Request	Request sai format hoặc thiếu field
401	Unauthorized	Chưa đăng nhập hoặc token sai
403	Forbidden	Đăng nhập rồi nhưng không đủ quyền
404	Not Found	Không tìm thấy resource
409	Conflict	Trùng dữ liệu hoặc sai trạng thái hiện tại
422	Business Rule Failed	Vi phạm rule nghiệp vụ
500	Internal Server Error	Lỗi server
502	Provider Error	Lỗi từ PayOS/SePay hoặc service ngoài

5. Global Try-Catch Policy

5.1 Controller Layer

Controller chỉ làm:

- Nhận request
- Validate input cơ bản
- Gọi service/use case
- Trả response

Controller không viết business logic.

5.2 Service Layer

Service xử lý:

- Business rule
 - Transaction
 - Validate quyền sở hữu dữ liệu
 - Gọi repository
 - Gọi provider như PayOS/SePay
 - Publish hoặc consume Kafka event
-

5.3 Exception Mapping

Exception	Status	Example
ValidationException	400	Email required
UnauthorizedException	401	Invalid token
ForbiddenException	403	Role not allowed
NotFoundException	404	Product not found

ConflictException	409	Email already exists
BusinessRuleException	422	Design must be saved before Try-On
ProviderException	502	PayOS/SePay error
ServiceUnavailableException	503	Payment provider unavailable
UnexpectedException	500	Unknown error

6. API Purpose Summary

6.1 C# API Summary

API	Purpose	Used By	Screen
POST /auth/register	Tạo tài khoản khách hàng mới	Guest	Register Page
POST /auth/login	Đăng nhập và lấy JWT	Guest	Login Page
POST /auth/refresh-token	Lấy access token mới	FE App	Silent refresh
POST /auth/logout	Thu hồi refresh token	Logged-in user	Logout button
GET /auth/me	Lấy thông tin user hiện tại	FE App	Header/Profile
POST /payments	Tạo payment qua PayOS/SePay	Customer	Checkout Page
GET /payments/order/{orderId}	Lấy trạng thái thanh toán của order	Customer/Staff/Admin	Payment Status Page
POST /payments/webhooks/payos	Nhận kết quả thanh toán từ PayOS	PayOS Provider	Provider Webhook
POST /payments/webhooks/sepay	Nhận kết quả chuyển khoản từ SePay	SePay Provider	Provider Webhook

GET /payments/{paymentId}/invoice	Lấy hóa đơn PDF sau thanh toán	Customer/Staff/Admin	Order Detail
GET /about-us	Hiển thị nội dung About Us	Guest/Customer	About Us Page
PUT /admin/about-us/{sectionKey}	Cập nhật nội dung About Us	Staff/Admin	Admin Content Page
GET /staff/orders	Staff xem danh sách đơn cần xử lý	Staff/Admin	Staff Order Page
GET /staff/orders/{orderId}/print-info	Staff lấy thông tin file in	Staff/Admin	Print Detail Page

6.2 Java API Summary

API	Purpose	Used By	Screen
GET /products	Hiển thị danh sách sản phẩm	Guest/Customer	Product Catalog
GET /products/{productId}	Xem chi tiết sản phẩm và variant	Guest/Customer	Product Detail
POST /admin/products	Admin/Staff tạo sản phẩm	Staff/Admin	Product Management
PATCH /admin/product-variants/{variantId}/inventory	Cập nhật tồn kho	Staff/Admin	Inventory Page
POST /designs	Tạo design draft	Customer	Design Editor
PUT /designs/{designId}/save	Lưu thiết kế chính thức	Customer	Design Editor
GET /designs/my	Xem thiết kế của tôi	Customer	My Designs
GET /designs/{designId}	Xem chi tiết design	Customer/Staff/Admin	Design Detail

POST /tryon/requests	Gửi yêu cầu AI Try-On	Customer	Try-On Page
GET /tryon/requests/{requestId}	Kiểm tra trạng thái Try-On	Customer	Try-On Loading
GET /tryon/requests/{requestId}/result	Lấy ảnh Try-On kết quả	Customer	Try-On Result
POST /orders	Tạo đơn hàng từ design	Customer	Checkout Page
GET /orders/my	Xem đơn hàng của tôi	Customer	My Orders
GET /orders/{orderId}	Xem chi tiết đơn hàng	Customer/Staff/Admin	Order Detail
PATCH /staff/orders/{orderId}/status	Cập nhật trạng thái đơn	Staff/Admin	Staff Operation
POST /feedbacks	Customer gửi feedback	Customer	Feedback Form
GET /feedbacks/public	Xem feedback công khai	Guest/Customer	Product Detail/About
PATCH /admin/feedbacks/{feedbackId}/status	Duyệt feedback	Staff/Admin	Feedback Moderation

7. Domain 1 — Authentication API

Owner: C# .NET 8

Tables: users, roles, user_roles, refresh_tokens

7.1 Register

POST /api/auth/register

API Purpose

API này dùng để tạo tài khoản mới cho khách hàng.

Frontend gọi API này khi user điền form đăng ký. Sau khi register thành công, hệ thống tạo user, hash password và gán role mặc định là **CUSTOMER**.

Used By

User	Screen
------	--------

Guest	Register Page
-------	---------------

Request

```
{
  "email": "customer@example.com",
  "password": "Password@123",
  "fullName": "Nguyen Van A",
  "phone": "0909000000"
}
```

Success Response — 201

```
{
  "success": true,
  "message": "Register successfully",
  "data": {
    "userId": "uuid",
    "email": "customer@example.com",
    "fullName": "Nguyen Van A",
    "role": "CUSTOMER"
  },
  "errors": null,
  "meta": {
    "requestId": "req_001",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Email missing	400	EMAIL_REQUIRED
Password weak	400	PASSWORD_WEAK
Email already exists	409	EMAIL_ALREADY_EXISTS
Server error	500	INTERNAL_SERVER_ERROR

Backend Handling

Validate request
 Check email exists
 Hash password
 Create user
 Assign CUSTOMER role
 Return user information

7.2 Login

POST /api/auth/login

API Purpose

API này dùng để xác thực tài khoản và cấp access token cho user.

Frontend gọi API này ở màn Login. Nếu email/password đúng, backend trả về access token, refresh token và thông tin user để FE lưu session.

Used By

User	Screen
Guest	Login Page

Request

```
{
  "email": "customer@example.com",
  "password": "Password@123"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Login successfully",
  "data": {
    "accessToken": "jwt_access_token",
    "refreshToken": "refresh_token",
    "expiresIn": 3600,
    "user": {
      "id": "uuid",
      "email": "customer@example.com",
      "fullName": "Nguyen Van A",
      "roles": ["CUSTOMER"]
    }
  },
  "errors": null,
  "meta": {
    "requestId": "req_002",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Wrong email/password	401	INVALID_CREDENTIALS
User inactive	403	USER_INACTIVE
User banned	403	USER_BANNED

Backend Handling

Find user by email
Verify password hash
Check user status
Load roles
Generate access token
Generate refresh token
Store refresh token hash
Return token and user info

7.3 Refresh Token

POST /api/auth/refresh-token

API Purpose

API này dùng để lấy access token mới khi access token cũ hết hạn.

Frontend gọi API này tự động khi nhận lỗi 401 do access token expired.

Used By

User	Screen
Logged-in user	Silent refresh / App background

Request

```
{
  "refreshToken": "refresh_token"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Token refreshed",
  "data": {
    "accessToken": "new_access_token",
    "refreshToken": "new_refresh_token",
    "expiresIn": 3600
  },
  "errors": null,
  "meta": {
    "requestId": "req_003",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Missing token	400	REFRESH_TOKEN_REQUIRED
Expired token	401	REFRESH_TOKEN_EXPIRED
Revoked token	401	REFRESH_TOKEN_REVOKED

Backend Handling

Hash incoming refresh token
 Find refresh token hash in DB
 Check revoked_at is null
 Check expires_at
 Generate new access token
 Rotate refresh token
 Return new tokens

7.4 Logout

POST /api/auth/logout

API Purpose

API này dùng để đăng xuất user và thu hồi refresh token.

Frontend gọi khi user bấm Logout. Backend set **revoked_at** cho refresh token.

Used By

User	Screen
CUSTOMER/STAFF/ADMIN	Header / Profile Menu

Request

```
{
  "refreshToken": "refresh_token"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Logout successfully",
  "data": null,
  "errors": null,
  "meta": {
    "requestId": "req_004",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Backend Handling

Find refresh token
Set revoked_at
Return success

7.5 Get Current User

GET /api/auth/me

API Purpose

API này dùng để lấy thông tin user đang đăng nhập.

Frontend gọi khi app load lại, để biết user là CUSTOMER, STAFF hay ADMIN và hiển thị menu phù hợp.

Used By

User	Screen
CUSTOMER/STAFF/ADMIN	App initialization, Header, Profile

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "id": "uuid",
  }
}
```

```
"email": "customer@example.com",
"fullName": "Nguyen Van A",
"phone": "0909000000",
"avatarUrl": null,
"roles": ["CUSTOMER"],
"status": "ACTIVE"
},
"errors": null,
"meta": {
  "requestId": "req_005",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

8. Domain 2 — Catalog API

Owner: Java Spring Boot

Tables: products, product_variants, product_images, product_inventory

8.1 Get Product List

GET /api/products?keyword=tshirt&status=ACTIVE&page=1&pageSize=12

API Purpose

API này dùng để hiển thị danh sách sản phẩm áo cho Guest/Customer.

Frontend gọi API này ở Product Catalog Page hoặc Landing Page. API trả về sản phẩm, giá cơ bản và ảnh thumbnail.

Used By

User	Screen
Guest/Customer	Product Catalog, Landing Page

Success Response — 200

```
{
```

```
"success": true,
"message": "Success",
"data": {
  "items": [
    {
      "id": "uuid",
      "name": "Basic T-Shirt",
      "description": "Basic cotton t-shirt",
      "basePrice": 150000,
      "thumbnailUrl": "https://cdn.example.com/basic-shirt.png",
      "status": "ACTIVE"
    }
  ],
  "page": 1,
  "pageSize": 12,
  "totalItems": 30,
  "totalPages": 3
},
"errors": null,
"meta": {
  "requestId": "req_006",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Status & Error

Case	Status	Error Code
Invalid page	400	INVALID_PAGE
Invalid page size	400	INVALID_PAGE_SIZE

Backend Handling

Validate pagination
Filter only ACTIVE products for public user
Join thumbnail image
Return paginated product list

8.2 Get Product Detail

GET /api/products/{productId}

API Purpose

API này dùng để xem chi tiết một sản phẩm, bao gồm ảnh, size, màu, chất liệu và tồn kho.

Frontend gọi API này ở Product Detail Page trước khi user vào Design Editor.

Used By

User	Screen
Guest/Customer	Product Detail Page

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "id": "uuid",
    "name": "Basic T-Shirt",
    "description": "Basic cotton t-shirt",
    "basePrice": 150000,
    "status": "ACTIVE",
    "images": [
      {
        "id": "uuid",
        "imageUrl": "https://cdn.example.com/basic-shirt.png",
        "isThumbnail": true,
        "sortOrder": 1
      }
    ],
    "variants": [
      {
        "id": "uuid",
        "sku": "BASIC-WHITE-M",
        "size": "M",
        "color": "White",
        "material": "Cotton",
        "priceAdjustment": 0,
        "availableQuantity": 20,
        "status": "ACTIVE"
      }
    ]
  }
}
```

```
},
"errors": null,
"meta": {
  "requestId": "req_007",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Status & Error

Case	Status	Error Code
Product not found	404	PRODUCT_NOT_FOUND
Product inactive	404	PRODUCT_NOT_AVAILABLE

Backend Handling

Find product
Check product ACTIVE if public request
Load product images
Load variants
Load inventory by variant
Return product detail

8.3 Create Product

POST /api/admin/products

API Purpose

API này dùng để Admin/Staff tạo sản phẩm mới cho cửa hàng.

Frontend gọi API này ở Product Management Page. API tạo product, images, variants và inventory ban đầu.

Used By

User	Screen
------	--------

Request

```
{
  "name": "Oversize T-Shirt",
  "description": "Oversize cotton t-shirt",
  "basePrice": 190000,
  "status": "ACTIVE",
  "images": [
    {
      "imageUrl": "https://cdn.example.com/oversize.png",
      "isThumbnail": true,
      "sortOrder": 1
    }
  ],
  "variants": [
    {
      "sku": "OVERSIZE-WHITE-M",
      "size": "M",
      "color": "White",
      "material": "Cotton",
      "priceAdjustment": 20000,
      "availableQuantity": 30
    }
  ]
}
```

Success Response — 201

```
{
  "success": true,
  "message": "Product created",
  "data": {
    "productId": "uuid"
  },
  "errors": null,
  "meta": {
    "requestId": "req_008",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Missing name	400	PRODUCT_NAME_REQUIRED
Invalid price	400	INVALID_BASE_PRICE
Duplicate SKU	409	SKU_ALREADY_EXISTS
No permission	403	FORBIDDEN

Backend Handling

Validate ADMIN/STAFF role
Validate product data
Validate images
Validate variants
Check duplicate SKU
Create product
Create product_images
Create product_variants
Create product_inventory
Commit transaction

8.4 Update Inventory

PATCH /api/admin/product-variants/{variantId}/inventory

API Purpose

API này dùng để cập nhật tồn kho của một variant.

Frontend gọi API này ở Inventory Page. Staff/Admin dùng API này khi nhập thêm hàng hoặc chỉnh tồn kho.

Used By

User	Screen
ADMIN/STAFF	Inventory Management Page

Request

```
{
  "availableQuantity": 50,
  "reason": "Restock size M white"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Inventory updated",
  "data": {
    "productVariantId": "uuid",
    "availableQuantity": 50,
    "reservedQuantity": 0,
    "soldQuantity": 0
  },
  "errors": null,
  "meta": {
    "requestId": "req_009",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Variant not found	404	VARIANT_NOT_FOUND
Negative quantity	400	INVALID_QUANTITY
No permission	403	FORBIDDEN

Backend Handling

Validate role
Find variant
Find inventory row
Validate quantity ≥ 0
Update available_quantity
Return updated inventory

9. Domain 3 — Design API

Owner: Java Spring Boot

Tables: designs, design_layers

9.1 Create Draft Design

POST /api/designs

API Purpose

API này dùng để tạo bản nháp thiết kế trước khi user bắt đầu chỉnh áo trong editor.

Frontend gọi API này khi Customer chọn sản phẩm + variant rồi bấm “Start Design”.

Used By

User

Screen

CUSTOMER Product Detail → Design Editor

Request

```
{
  "productId": "uuid",
  "productVariantId": "uuid",
  "name": "My First Shirt"
}
```

Success Response — 201

```
{
  "success": true,
  "message": "Design draft created",
  "data": {
    "designId": "uuid",
    "status": "DRAFT"
  },
  "errors": null,
  "meta": {
    "requestId": "req_010",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

```
}  
}
```

Status & Error

Case	Status	Error Code
Product not found	404	PRODUCT_NOT_FOUND
Variant not found	404	VARIANT_NOT_FOUND
Variant inactive	422	VARIANT_NOT_AVAILABLE

Backend Handling

Validate CUSTOMER role
Check product exists
Check variant exists and active
Create design with status DRAFT
Return designId

9.2 Save Design

PUT /api/designs/{designId}/save

API Purpose

API này dùng để lưu thiết kế chính thức của Customer.

Frontend gọi API này khi user bấm “Save Design” trong Fabric.js editor. API lưu **canvasJson**, preview image, print file URL và layer data.

Used By

User	Screen
CUSTOMER	Design Editor

Request

```
{
```

```
"name": "My First Shirt",
"canvasJson": {
  "version": "5.3.0",
  "objects": []
},
"previewImageUrl": "https://cdn.example.com/design-preview.png",
"printFileUrl": "https://cdn.example.com/print-file.pdf",
"layers": [
  {
    "layerType": "TEXT",
    "content": "HELLO",
    "positionX": 120,
    "positionY": 180,
    "width": 200,
    "height": 60,
    "rotation": 0,
    "color": "#000000",
    "zIndex": 1
  }
]
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Design saved",
  "data": {
    "designId": "uuid",
    "status": "SAVED",
    "previewImageUrl": "https://cdn.example.com/design-preview.png",
    "printFileUrl": "https://cdn.example.com/print-file.pdf"
  },
  "errors": null,
  "meta": {
    "requestId": "req_011",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
------	--------	------------

Design not found	404	DESIGN_NOT_FOUND
Not owner	403	DESIGN_ACCESS_DENIED
Design locked	409	DESIGN_LOCKED
Invalid canvas	400	INVALID_CANVAS_JSON
Too many layers	422	DESIGN_LAYER_LIMIT_EXCEEDED

Backend Handling

Find design
 Check owner
 Check status is not LOCKED
 Validate canvasJson
 Validate layers
 Replace old layers
 Save new layers
 Update design status to SAVED
 Publish DesignSaved event
 Return design data

9.3 Get My Designs

GET /api/designs/my?page=1&pageSize=10

API Purpose

API này dùng để Customer xem lại các thiết kế đã tạo.

Frontend gọi API này ở My Designs Page hoặc khi Customer muốn chọn design cũ để Try-On hoặc đặt hàng.

Used By

User	Screen
CUSTOMER	My Designs Page

Success Response — 200

```
{
```

```
"success": true,
"message": "Success",
"data": {
  "items": [
    {
      "id": "uuid",
      "name": "My First Shirt",
      "productId": "uuid",
      "productVariantId": "uuid",
      "previewImageUrl": "https://cdn.example.com/design-preview.png",
      "status": "SAVED",
      "createdAt": "2026-06-19T10:30:00Z"
    }
  ],
  "page": 1,
  "pageSize": 10,
  "totalItems": 1,
  "totalPages": 1
},
"errors": null,
"meta": {
  "requestId": "req_012",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

9.4 Get Design Detail

GET /api/designs/{designId}

API Purpose

API này dùng để lấy đầy đủ dữ liệu của một design.

Customer dùng để mở lại design trong editor. Staff dùng để xem file in khi xử lý đơn.

Used By

User	Screen
CUSTOMER	Design Editor / My Designs
STAFF/ADMIN	Print Info / Order Detail

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "id": "uuid",
    "customerId": "uuid",
    "productId": "uuid",
    "productVariantId": "uuid",
    "name": "My First Shirt",
    "canvasJson": {},
    "previewImageUrl": "https://cdn.example.com/design-preview.png",
    "printFileUrl": "https://cdn.example.com/print-file.pdf",
    "status": "SAVED",
    "layers": []
  },
  "errors": null,
  "meta": {
    "requestId": "req_013",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Access Rule

Role	Rule
CUSTOMER	Chỉ xem design của chính mình
STAFF/ADMIN	Xem để xử lý đơn hoặc kiểm tra

10. Domain 4 — AI Try-On API

Owner: Java Spring Boot + AI Worker

Tables: tryon_requests, tryon_results

Kafka: TryOnRequested, TryOnCompleted, TryOnFailed

10.1 Request Try-On

POST /api/tryon/requests

API Purpose

API này dùng để tạo yêu cầu AI mặc thử áo.

Frontend gọi API này khi Customer bấm “Try-On”. API không xử lý AI ngay trong request mà tạo job async và gửi Kafka event.

Used By

User	Screen
CUSTOMER	Try-On Page

Request

```
{
  "designId": "uuid",
  "userPhotoUrl": "https://cdn.example.com/user-photo.png",
  "heightCm": 170,
  "weightKg": 60
}
```

Success Response — 202

```
{
  "success": true,
  "message": "Try-On request accepted",
  "data": {
    "tryonRequestId": "uuid",
    "status": "PENDING"
  },
  "errors": null,
  "meta": {
    "requestId": "req_014",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Design not found	404	DESIGN_NOT_FOUND

Design not saved	422	DESIGN_MUST_BE_SAVED
Not owner	403	DESIGN_ACCESS_DENIED
Invalid photo URL	400	INVALID_USER_PHOTO
AI unavailable	503	AI_SERVICE_UNAVAILABLE

Backend Handling

Validate CUSTOMER role
 Find design
 Check design belongs to customer
 Check design status SAVED or LOCKED
 Create tryon_request with PENDING
 Publish TryOnRequested event
 Return 202 Accepted

10.2 Get Try-On Request Status

GET /api/tryon/requests/{requestId}

API Purpose

API này dùng để kiểm tra AI xử lý tới đâu.

Frontend gọi API này bằng polling ở màn loading Try-On. Nếu status là **SUCCEEDED**, FE gọi API lấy result.

Used By

User	Screen
CUSTOMER	Try-On Loading Page

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "id": "uuid",
    "designId": "uuid",
```

```
{
  "status": "PROCESSING",
  "errorMessage": null,
  "requestedAt": "2026-06-19T10:30:00Z",
  "completedAt": null
},
"errors": null,
"meta": {
  "requestId": "req_015",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

10.3 Get Try-On Result

GET /api/tryon/requests/{requestId}/result

API Purpose

API này dùng để lấy ảnh kết quả mặc thử sau khi AI xử lý xong.

Frontend gọi API này khi request status là **SUCCEEDED**.

Used By

User	Screen
CUSTOMER	Try-On Result Page

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "tryonRequestId": "uuid",
    "designId": "uuid",
    "resultImageUrl": "https://cdn.example.com/tryon-result.png",
    "processingTimeMs": 8500,
    "createdAt": "2026-06-19T10:30:00Z"
  },
  "errors": null,
  "meta": {
```

```
"requestId": "req_016",
"timestamp": "2026-06-19T10:30:00Z"
}
}
```

Status & Error

Case	Status	Error Code
Request not found	404	TRYON_REQUEST_NOT_FOUND
Result not ready	202	TRYON_RESULT_NOT_READY
Try-On failed	422	TRYON_FAILED

11. Domain 5 — Order API

Owner: Java Spring Boot

Tables: orders, order_items, order_status_history

Kafka: OrderCreated, OrderPaid, OrderCompleted

11.1 Create Order

POST /api/orders

API Purpose

API này dùng để tạo đơn hàng từ một hoặc nhiều design đã lưu.

Frontend gọi API này ở Checkout Page trước khi chuyển sang payment. API kiểm tra tồn kho, tính tiền, tạo order và phát event **OrderCreated**.

Used By

User	Screen
CUSTOMER	Checkout Page

Request

```
{
  "items": [
    {
      "productId": "uuid",
      "productVariantId": "uuid",
      "designId": "uuid",
      "quantity": 1
    }
  ],
  "receiverName": "Nguyen Van A",
  "receiverPhone": "0909000000",
  "shippingAddress": "123 Nguyen Trai, Quan 1, TP.HCM"
}
```

Success Response — 201

```
{
  "success": true,
  "message": "Order created",
  "data": {
    "orderId": "uuid",
    "orderCode": "ORD202606190001",
    "totalAmount": 180000,
    "paymentStatus": "PENDING",
    "orderStatus": "PENDING_PAYMENT"
  },
  "errors": null,
  "meta": {
    "requestId": "req_017",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Empty items	400	ORDER_ITEMS_REQUIRED
Product not found	404	PRODUCT_NOT_FOUND
Variant not found	404	VARIANT_NOT_FOUND

Design not found	404	DESIGN_NOT_FOUND
Design not saved	422	DESIGN_MUST_BE_SAVED
Not design owner	403	DESIGN_ACCESS_DENIED
Out of stock	422	PRODUCT_OUT_OF_STOCK
Invalid quantity	400	INVALID_QUANTITY

Backend Handling

Validate CUSTOMER role

Validate request items

For each item:

Check product active

Check variant active

Check inventory

Check design owner

Check design status SAVED

Calculate total amount

Create order

Create order_items with snapshot

Insert order_status_history

Publish OrderCreated event

Return order data

11.2 Get My Orders

GET /api/orders/my?page=1&pageSize=10

API Purpose

API này dùng để Customer xem danh sách đơn hàng của mình.

Frontend gọi API này ở My Orders Page.

Used By

User	Screen
CUSTOMER	My Orders Page

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "items": [
      {
        "id": "uuid",
        "orderCode": "ORD202606190001",
        "totalAmount": 180000,
        "paymentStatus": "PENDING",
        "orderStatus": "PENDING_PAYMENT",
        "createdAt": "2026-06-19T10:30:00Z"
      }
    ],
    "page": 1,
    "pageSize": 10,
    "totalItems": 1,
    "totalPages": 1
  },
  "errors": null,
  "meta": {
    "requestId": "req_018",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

11.3 Get Order Detail

GET /api/orders/{orderId}

API Purpose

API này dùng để xem chi tiết đơn hàng.

Customer dùng để theo dõi đơn. Staff dùng để xử lý sản xuất. Admin dùng để kiểm tra đơn.

Used By

User	Screen
CUSTOMER	Order Detail

STAFF	Staff Order Detail
ADMIN	Admin Order Management

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "id": "uuid",
    "orderCode": "ORD202606190001",
    "customerId": "uuid",
    "totalAmount": 180000,
    "paymentStatus": "PAID",
    "orderStatus": "IN_PRODUCTION",
    "receiverName": "Nguyen Van A",
    "receiverPhone": "0909000000",
    "shippingAddress": "123 Nguyen Trai, Quan 1, TP.HCM",
    "items": [
      {
        "id": "uuid",
        "productId": "uuid",
        "productVariantId": "uuid",
        "designId": "uuid",
        "productNameSnapshot": "Basic T-Shirt",
        "variantSnapshot": {
          "size": "M",
          "color": "White",
          "material": "Cotton"
        },
        "quantity": 1,
        "unitPrice": 180000,
        "totalPrice": 180000
      }
    ],
    "statusHistory": []
  },
  "errors": null,
  "meta": {
    "requestId": "req_019",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Access Rule

Role	Rule
CUSTOMER	Chỉ xem order của chính mình
STAFF	Xem order để xử lý
ADMIN	Xem tất cả order

11.4 Update Order Status

PATCH /api/staff/orders/{orderId}/status

API Purpose

API này dùng để Staff cập nhật trạng thái xử lý đơn hàng.

Frontend gọi API này ở Staff Order Detail Page khi Staff chuyển đơn từ **PAID** sang **IN_PRODUCTION**, **SHIPPING**, hoặc **COMPLETED**.

Used By

User	Screen
STAFF/ADMIN	Staff Order Detail

Request

```
{
  "toStatus": "IN_PRODUCTION",
  "note": "Start printing process"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Order status updated",
  "data": {
    "orderId": "uuid",
    "fromStatus": "PAID",
```

```
"toStatus": "IN_PRODUCTION"
},
"errors": null,
"meta": {
  "requestId": "req_020",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Allowed Transitions

From	To
PAID	IN_PRODUCTION
IN_PRODUCTION	SHIPPING
SHIPPING	COMPLETED
PAID	CANCELLED

Status & Error

Case	Status	Error Code
Order not found	404	ORDER_NOT_FOUND
Payment not paid	422	ORDER_NOT_PAID
Invalid transition	409	INVALID_ORDER_STATUS_TRANSITION
No permission	403	FORBIDDEN

Backend Handling

Validate STAFF/ADMIN role
Find order
Check payment_status PAID
Validate status transition
Update order_status
Insert order_status_history
If status COMPLETED:
 Publish OrderCompleted event
Return updated status

12. Domain 6 — Payment API

Owner: C# .NET 8

Tables: payments

Provider: PayOS or SePay

Kafka: PaymentSucceeded, PaymentFailed

12.1 Create Payment

POST /api/payments

API Purpose

API này dùng để tạo giao dịch thanh toán thật cho một order.

Frontend gọi API này sau khi order đã được tạo. C# Payment Service sẽ tạo payment record, gọi PayOS hoặc SePay, sau đó trả về link thanh toán hoặc QR thanh toán cho Customer.

Used By

User	Screen
CUSTOMER	Checkout Payment Page

Request

```
{
  "orderId": "uuid",
  "provider": "PAYOS",
  "returnUrl": "http://localhost:5173/payment/success",
  "cancelUrl": "http://localhost:5173/payment/cancel"
}
```

Hoặc:

```
{
  "orderId": "uuid",
  "provider": "SEPAY",
}
```

```
"returnUrl": "http://localhost:5173/payment/success",
"cancelUrl": "http://localhost:5173/payment/cancel"
}
```

Success Response — PayOS — 201

```
{
  "success": true,
  "message": "Payment created",
  "data": {
    "paymentId": "uuid",
    "orderId": "uuid",
    "provider": "PAYOS",
    "amount": 180000,
    "paymentStatus": "PENDING",
    "checkoutUrl": "https://pay.provider/checkout/xxx",
    "qrCodeUrl": "https://pay.provider/qr/xxx",
    "paymentContent": "ORD202606190001",
    "expiresAt": "2026-06-19T11:00:00Z"
  },
  "errors": null,
  "meta": {
    "requestId": "req_021",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Success Response — SePay — 201

```
{
  "success": true,
  "message": "Payment instruction created",
  "data": {
    "paymentId": "uuid",
    "orderId": "uuid",
    "provider": "SEPAY",
    "amount": 180000,
    "paymentStatus": "PENDING",
    "qrCodeUrl": "https://cdn.example.com/sepay-qr.png",
    "paymentContent": "ORD202606190001",
    "bankInfo": {
      "bankName": "Example Bank",
      "accountNumber": "123456789",
      "accountName": "AI TSHIRT STORE"
    }
  },
  "errors": null,
  "meta": {
    "requestId": "req_021",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

```
"expiresAt": "2026-06-19T11:00:00Z"
},
"errors": null,
"meta": {
  "requestId": "req_022",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Status & Error

Case	Status	Error Code
Order not found	404	ORDER_NOT_FOUND
Order not owned	403	ORDER_ACCESS_DENIED
Order already paid	409	ORDER_ALREADY_PAID
Payment already exists	409	PAYMENT_ALREADY_EXISTS
Invalid provider	400	INVALID_PAYMENT_PROVIDER
Provider unavailable	503	PAYMENT_PROVIDER_UNAVAILABLE
Provider error	502	PAYMENT_PROVIDER_ERROR

Backend Handling

Validate CUSTOMER role
Call Java Order API to get order payment info
Check order belongs to customer
Check order status PENDING_PAYMENT
Check payment not already PAID
Create payment record PENDING
If provider PAYOS:
 Call PayOS adapter
 Save checkout_url, qr_code_url, provider_payment_id
If provider SEPAY:
 Generate payment_content
 Generate QR/payment instruction
 Save qr_code_url and payment_content
Return payment information

12.2 Get Payment By Order

GET /api/payments/order/{orderId}

API Purpose

API này dùng để kiểm tra trạng thái thanh toán của một order.

Frontend gọi API này ở Payment Status Page hoặc Order Detail Page để biết order đã thanh toán chưa.

Used By

User	Screen
CUSTOMER	Payment Status / Order Detail
STAFF/ADMIN	Order Management

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "paymentId": "uuid",
    "orderId": "uuid",
    "customerId": "uuid",
    "amount": 180000,
    "paymentMethod": "PAYOS",
    "paymentStatus": "PENDING",
    "providerPaymentId": "provider_payment_id",
    "transactionCode": null,
    "checkoutUrl": "https://pay.provider/checkout/xxx",
    "qrCodeUrl": "https://pay.provider/qr/xxx",
    "paymentContent": "ORD202606190001",
    "invoiceNumber": null,
    "invoicePdfUrl": null,
    "paidAt": null
  },
  "errors": null,
  "meta": {
    "requestId": "req_023",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

```
}
```

12.3 PayOS Webhook

POST /api/payments/webhooks/payos

API Purpose

API này dùng để nhận thông báo thanh toán từ PayOS.

Frontend không gọi API này. Đây là endpoint để PayOS gọi về hệ thống khi Customer thanh toán thành công, thất bại hoặc hủy thanh toán.

Used By

Caller	Screen
PayOS Provider	No UI

Request

```
{
  "providerPaymentId": "provider_payment_id",
  "orderCode": "ORD202606190001",
  "amount": 180000,
  "status": "PAID",
  "transactionCode": "FT123456",
  "paidAt": "2026-06-19T10:30:00Z",
  "signature": "provider_signature"
}
```

Success Response — 200

```
{
  "success": true
}
```

Webhook Handling Policy

Case	HTTP Status	Internal Action
Valid payment	200	Update payment PAID, publish PaymentSucceeded
Duplicate webhook	200	Ignore safely
Invalid signature	400	Reject webhook
Payment not found	200	Log unmatched webhook
Amount mismatch	200	Mark suspicious, do not publish event
Temporary DB error	500	Allow provider retry

Backend Handling

Receive webhook
 Verify provider signature
 Find payment by providerPaymentId or orderCode
 If payment already PAID:
 Return 200
 Validate amount
 Update payment_status = PAID
 Save transaction_code
 Set paid_at
 Generate invoice_number
 Generate invoice_pdf_url
 Publish PaymentSucceeded event
 Return 200

12.4 SePay Webhook

POST /api/payments/webhooks/sepay

API Purpose

API này dùng để nhận webhook từ SePay khi có giao dịch chuyển khoản vào tài khoản ngân hàng.

Frontend không gọi API này. Đây là endpoint cho SePay gọi về hệ thống để tự động xác nhận thanh toán.

Used By

Caller	Screen
SePay Provider	No UI

Request

```
{
  "transactionId": "sepay_transaction_id",
  "gateway": "Vietcombank",
  "transactionDate": "2026-06-19 10:30:00",
  "accountNumber": "123456789",
  "transactionCode": "FT123456",
  "content": "ORD202606190001",
  "transferType": "in",
  "transferAmount": 180000,
  "referenceCode": "REF123456",
  "description": "Payment for ORD202606190001"
}
```

Success Response — 200

```
{
  "success": true
}
```

Webhook Handling Policy

Case	HTTP Status	Internal Action
Valid transfer	200	Update payment PAID, publish PaymentSucceeded
Duplicate transaction	200	Ignore safely
Invalid auth/signature	400	Reject webhook
Payment content not matched	200	Log unmatched transaction
Amount mismatch	200	Do not mark PAID
Temporary DB error	500	Allow retry

Backend Handling

Receive webhook
Verify SePay auth/signature/API key
Check transferType is incoming
Extract payment_content from transaction content
Find payment by payment_content
If payment already PAID:
 Return 200
Validate transferAmount == payment.amount
Update payment_status = PAID
Save transaction_code
Set paid_at
Generate invoice_number
Generate invoice_pdf_url
Publish PaymentSucceeded event
Return 200

12.5 Get Invoice PDF

GET /api/payments/{paymentId}/invoice

API Purpose

API này dùng để lấy hóa đơn PDF sau khi thanh toán thành công.

Customer dùng để tải hóa đơn. Staff/Admin dùng để kiểm tra hóa đơn khi xử lý đơn.

Used By

User	Screen
CUSTOMER	Order Detail
STAFF/ADMIN	Staff Order Detail

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "invoiceNumber": "INV202606190001",
    "invoicePdfUrl": "https://cdn.example.com/invoices/INV202606190001.pdf"
  }
}
```

```
},
"errors": null,
"meta": {
  "requestId": "req_024",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Status & Error

Case	Status	Error Code
Payment not found	404	PAYMENT_NOT_FOUND
Payment not paid	422	PAYMENT_NOT_PAID
Invoice not generated	404	INVOICE_NOT_FOUND

13. Domain 7 — Feedback API

Owner: Java Spring Boot

Tables: feedbacks

13.1 Submit Feedback

POST /api/feedbacks

API Purpose

API này dùng để Customer gửi đánh giá sau khi đơn hàng hoàn tất.

Frontend gọi API này ở Feedback Form trong Order Detail Page.

Used By

User	Screen
CUSTOMER	Order Detail / Feedback Form

Request

```
{
  "orderId": "uuid",
  "productId": "uuid",
  "rating": 5,
  "comment": "Áo đẹp, chất lượng tốt",
  "imageUrl": "https://cdn.example.com/feedback.png"
}
```

Success Response — 201

```
{
  "success": true,
  "message": "Feedback submitted",
  "data": {
    "feedbackId": "uuid",
    "status": "PENDING"
  },
  "errors": null,
  "meta": {
    "requestId": "req_025",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Status & Error

Case	Status	Error Code
Order not found	404	ORDER_NOT_FOUND
Order not completed	422	ORDER_NOT_COMPLETED
Invalid rating	400	INVALID_RATING
Product not in order	422	PRODUCT_NOT_IN_ORDER

Backend Handling

Validate CUSTOMER role
Find order
Check order belongs to customer
Check order status COMPLETED

Check product exists in order_items
Validate rating 1-5
Create feedback with PENDING
Return feedbackId

13.2 Get Public Feedbacks

GET /api/feedbacks/public?productId=uuid&page=1&pageSize=10

API Purpose

API này dùng để hiển thị feedback công khai cho Guest/Customer.

Frontend gọi API này ở Product Detail Page hoặc Landing Page để tăng độ tin cậy.

Used By

User	Screen
Guest/Customer	Product Detail / Landing Page

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "items": [
      {
        "id": "uuid",
        "productId": "uuid",
        "rating": 5,
        "comment": "Áo đẹp",
        "imageUrl": "https://cdn.example.com/feedback.png",
        "createdAt": "2026-06-19T10:30:00Z"
      }
    ],
    "page": 1,
    "pageSize": 10,
    "totalItems": 1,
    "totalPages": 1
  },
  "errors": null,
```



```
"meta": {
  "requestId": "req_026",
  "timestamp": "2026-06-19T10:30:00Z"
}
```

Backend Handling

Filter feedback status APPROVED
Optional filter by productId
Return paginated feedback

13.3 Moderate Feedback

PATCH /api/admin/feedbacks/{feedbackId}/status

API Purpose

API này dùng để Staff/Admin duyệt, ẩn hoặc từ chối feedback.

Frontend gọi API này ở Feedback Moderation Page.

Used By

User	Screen
STAFF/ADMIN	Feedback Management Page

Request

```
{
  "status": "APPROVED",
  "note": "Valid feedback"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "Feedback status updated",
}
```

```
"data": {
  "feedbackId": "uuid",
  "status": "APPROVED"
},
"errors": null,
"meta": {
  "requestId": "req_027",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Backend Handling

Validate STAFF/ADMIN role
Find feedback
Validate target status
Update feedback status
Set reviewed_by
Return updated status

14. Domain 8 — About Us API

Owner: C# Content/Admin Service
Tables: about_us_contents

14.1 Get Published About Us

GET /api/about-us

API Purpose

API này dùng để hiển thị nội dung giới thiệu nền tảng cho Guest/Customer.

Frontend gọi API này ở About Us Page hoặc Landing Page.

Used By

User

Screen

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": [
    {
      "sectionKey": "INTRODUCTION",
      "title": "Về nền tảng",
      "content": "Nền tảng thiết kế áo thun cá nhân hóa...",
      "imageUrl": "https://cdn.example.com/about.png"
    },
    {
      "sectionKey": "MISSION",
      "title": "Sứ mệnh",
      "content": "Giúp khách hàng tự tạo sản phẩm cá nhân hóa...",
      "imageUrl": null
    }
  ],
  "errors": null,
  "meta": {
    "requestId": "req_028",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Backend Handling

Find about_us_contents where status = PUBLISHED

Order by section_key or configured order

Return public content

14.2 Upsert About Us Section

PUT /api/admin/about-us/{sectionKey}

API Purpose

API này dùng để Staff/Admin tạo hoặc cập nhật một section trong About Us.

Frontend gọi API này ở Admin Content Management Page.

Used By

User	Screen
STAFF/ADMIN	About Us Management Page

Request

```
{
  "title": "Cách hoạt động",
  "content": "Chọn áo, thiết kế, thử bằng AI và đặt hàng.",
  "imageUrl": "https://cdn.example.com/how-it-works.png",
  "status": "PUBLISHED"
}
```

Success Response — 200

```
{
  "success": true,
  "message": "About Us section updated",
  "data": {
    "sectionKey": "HOW_IT_WORKS",
    "status": "PUBLISHED"
  },
  "errors": null,
  "meta": {
    "requestId": "req_029",
    "timestamp": "2026-06-19T10:30:00Z"
  }
}
```

Backend Handling

Validate STAFF/ADMIN role
Validate sectionKey
Find existing section
If exists:
 update content
If not exists:
 create section
Save status DRAFT/PUBLISHED
Return sectionKey and status

15. Domain 9 — Staff Operation API

Owner: C# Operation Gateway

Purpose: Validate STAFF/ADMIN role, then call Java APIs

15.1 Staff Get Paid Orders

GET /api/staff/orders?status=PAID&page=1&pageSize=10

API Purpose

API này dùng để Staff xem danh sách đơn hàng cần xử lý.

C# Operation Gateway nhận request từ Staff UI, kiểm tra role, rồi gọi Java Order API để lấy dữ liệu đơn.

Used By

User	Screen
STAFF/ADMIN	Staff Order List

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "items": [
      {
        "orderId": "uuid",
        "orderCode": "ORD202606190001",
        "customerId": "uuid",
        "totalAmount": 180000,
        "orderStatus": "PAID",
        "paymentStatus": "PAID"
      }
    ],
    "page": 1,
  }
}
```

```
"pageSize": 10,
"totalItems": 1,
"totalPages": 1
},
"errors": null,
"meta": {
  "requestId": "req_030",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Backend Handling

C# validates JWT
C# checks STAFF/ADMIN role
C# calls Java GET /api/orders/internal
C# maps response
Return to Staff UI

15.2 Staff Get Print Info

GET /api/staff/orders/{orderId}/print-info

API Purpose

API này dùng để Staff lấy thông tin cần thiết để in áo.

API trả về product snapshot, variant, quantity, design preview và print file URL.

Used By

User	Screen
STAFF/ADMIN	Print Detail Page

Success Response — 200

```
{
  "success": true,
  "message": "Success",
  "data": {
    "orderId": "uuid",
```

```
"orderCode": "ORD202606190001",
"items": [
  {
    "orderItemId": "uuid",
    "productName": "Basic T-Shirt",
    "variant": {
      "size": "M",
      "color": "White",
      "material": "Cotton"
    },
    "quantity": 1,
    "designId": "uuid",
    "previewImageUrl": "https://cdn.example.com/design-preview.png",
    "printFileUrl": "https://cdn.example.com/print-file.pdf"
  }
],
"errors": null,
"meta": {
  "requestId": "req_031",
  "timestamp": "2026-06-19T10:30:00Z"
}
}
```

Backend Handling

C# validates STAFF/ADMIN role
C# calls Java Order API
C# calls Java Design API if needed
C# combines print information
Return print package to Staff UI

16. Kafka Event Contract

16.1 OrderCreated

Purpose

Event này báo cho C# Payment Service biết rằng một order mới đã được tạo và cần tạo payment.

Producer

Java Order Service

Consumer

C# Payment Service

Payload

```
{
  "eventId": "uuid",
  "eventType": "OrderCreated",
  "occurredAt": "2026-06-19T10:30:00Z",
  "data": {
    "orderId": "uuid",
    "orderCode": "ORD202606190001",
    "customerId": "uuid",
    "totalAmount": 180000
  }
}
```

16.2 PaymentSucceeded

Purpose

Event này báo cho Java Order Service biết rằng payment đã thành công.

Java sẽ cập nhật order thành **PAID**, khóa design và ghi lịch sử trạng thái.

Producer

C# Payment Service

Consumer

Java Order Service

Payload

```
{
  "eventId": "uuid",
```



```
"eventType": "PaymentSucceeded",
"occurredAt": "2026-06-19T10:30:00Z",
"data": {
  "paymentId": "uuid",
  "orderId": "uuid",
  "customerId": "uuid",
  "amount": 180000,
  "provider": "PAYOS",
  "transactionCode": "FT123456",
  "paidAt": "2026-06-19T10:30:00Z",
  "invoiceNumber": "INV202606190001",
  "invoicePdfUrl": "https://cdn.example.com/invoices/INV202606190001.pdf"
}
}
```

16.3 TryOnRequested

Purpose

Event này báo cho AI Worker biết rằng Customer vừa yêu cầu AI Try-On.

Producer

Java Try-On Service

Consumer

AI Worker

Payload

```
{
  "eventId": "uuid",
  "eventType": "TryOnRequested",
  "occurredAt": "2026-06-19T10:30:00Z",
  "data": {
    "tryonRequestId": "uuid",
    "customerId": "uuid",
    "designId": "uuid",
    "userPhotoUrl": "https://cdn.example.com/user-photo.png"
  }
}
```

16.4 TryOnCompleted

Purpose

Event này báo cho Java Try-On Service biết AI đã xử lý xong ảnh Try-On.

Producer

AI Worker

Consumer

Java Try-On Service

Payload

```
{
  "eventId": "uuid",
  "eventType": "TryOnCompleted",
  "occurredAt": "2026-06-19T10:30:00Z",
  "data": {
    "tryonRequestId": "uuid",
    "designId": "uuid",
    "resultImageUrl": "https://cdn.example.com/tryon-result.png",
    "processingTimeMs": 8500
  }
}
```

17. Final API Design Summary

API doc này đã bổ sung rõ:

- API dùng để làm gì
- Ai gọi API
- Gọi ở màn hình nào
- Request body
- Success response
- Status code
- Error code
- Try/catch handling

- Table liên quan
- Kafka event liên quan
- C# và Java chia việc rõ

Final ownership:

Sid e	Main API Scope
C#	Auth, real payment PayOS/SePay, webhook, invoice PDF, About Us, Staff Gateway
Java	Product, Design, Try-On, Order, Feedback

Payment không còn là mock-only.

Payment Service thiết kế theo hướng:

Provider-ready: PAYOS or SEPAY

Java không update payment trực tiếp.

C# không update order trực tiếp.

Flow chuẩn:

Java creates order

↓

C# creates and confirms payment

↓

C# publishes PaymentSucceeded

↓

Java updates order and locks design

04-FrontEnd Spec

04. FRONTEND SPECIFICATION DOC

AI T-Shirt Customization & Virtual Try-On Platform

Version: 2.0

Based on: PRD Lite + DB 18 Tables + Backend API Spec

Frontend Team: 2 members

Frontend Level: Fresher

Frontend Stack: React + TypeScript + Tailwind CSS

Canvas Tool: Fabric.js basic

API State: TanStack Query basic

Client State: Zustand minimal

Routing: React Router

MVP Roles: ADMIN, STAFF, CUSTOMER

1. Document Purpose

Tài liệu này mô tả frontend scope cho MVP của hệ thống **AI T-Shirt Customization & Virtual Try-On Platform**.

Vì frontend team chỉ có **2 người** và đang ở mức **fresher**, tài liệu này tập trung vào:

- Làm được demo end-to-end
 - Không làm UI quá phức tạp
 - Không over-engineering state management
 - Gọi API đúng theo Backend API Spec
 - Matching với PRD Lite
 - Có Design Editor cơ bản bằng Fabric.js
 - Có payment flow với PayOS hoặc SePay ở mức frontend integration
 - Có Staff/Admin page đơn giản để demo vận hành
-

2. Frontend Level Target

Frontend level target:

Fresher cứng

Frontend không đặt mục tiêu junior/middle trong MVP.

Frontend cần làm tốt:

- React component cơ bản
- TypeScript type cơ bản
- React Router
- Form submit
- API call bằng Axios
- Loading state
- Error state
- Empty state
- Protected route
- Role-based redirect cơ bản
- Fabric.js canvas cơ bản
- Payment redirect/polling cơ bản
- Chia folder rõ ràng

Frontend chưa cần làm:

- Design system phức tạp
 - Animation phức tạp
 - Drag/drop layer nâng cao
 - State machine
 - Complex caching
 - Micro-frontend
 - Advanced performance optimization
 - Advanced admin dashboard
-

3. Frontend MVP Goal

Frontend MVP cần chứng minh được 6 flow chính:

Flow 1: Public Browsing

Landing Page

→ Product Catalog

→ Product Detail

→ About Us

→ Public Feedback

Flow 2: Authentication

Register

- Login
- Save token
- Redirect by role

Flow 3: Customer Design

Select Product

- Select Variant
- Create Design
- Open Design Editor
- Add Text/Image
- Save Design

Flow 4: AI Try-On

Saved Design

- Upload User Photo
- Request Try-On
- Polling Status
- Show Try-On Result

Flow 5: Order & Payment

Checkout

- Create Order
- Create Payment
- PayOS/SePay Payment UI
- Payment Success
- Order Detail

Flow 6: Staff Operation

Staff Login

- View Paid Orders
- View Print Info
- Update Order Status

4. Technology Stack

Technology	Usage Level	Purpose
React	Basic	Build UI
TypeScript	Basic	Type safety
Tailwind CSS	Basic	Styling
React Router	Basic	Routing
Axios	Basic	API calls
TanStack Query	Basic	GET API cache/loading
Zustand	Minimal	Auth token + editor state
Fabric.js	Basic	T-shirt design canvas
React Hook Form	Optional	Form handling
Zod	Optional	Form validation
Vite	Basic	Project setup/build

5. Frontend Architecture

5.1 Recommended Folder Structure

Dùng folder structure vừa đủ, không quá phức tạp.

```
src/  
  app/  
    App.tsx  
    router.tsx  
    providers.tsx  
  
  shared/  
    api/  
      httpClient.ts  
      apiResponse.ts  
    components/  
      Button.tsx  
      Input.tsx  
      Card.tsx  
      Modal.tsx  
      Loading.tsx
```


- EmptyState.tsx
- ErrorState.tsx
- ProtectedRoute.tsx
- RoleGuard.tsx
- constants/
 - roles.ts
 - routes.ts
 - orderStatus.ts
 - paymentStatus.ts
- utils/
 - formatCurrency.ts
 - formatDate.ts

- features/
 - auth/
 - api.ts
 - types.ts
 - authStore.ts
 - LoginPage.tsx
 - RegisterPage.tsx

- catalog/
 - api.ts
 - types.ts
 - ProductListPage.tsx
 - ProductDetailPage.tsx
 - ProductCard.tsx

- design/
 - api.ts
 - types.ts
 - designStore.ts
 - DesignEditorPage.tsx
 - MyDesignsPage.tsx
 - TShirtCanvas.tsx
 - EditorToolbar.tsx

- tryon/
 - api.ts
 - types.ts
 - TryOnPage.tsx
 - TryOnResultPage.tsx

- order/
 - api.ts
 - types.ts
 - CheckoutPage.tsx
 - MyOrdersPage.tsx

OrderDetailPage.tsx

payment/

api.ts

types.ts

PaymentPage.tsx

PaymentSuccessPage.tsx

PaymentCancelPage.tsx

feedback/

api.ts

types.ts

SubmitFeedbackPage.tsx

PublicFeedbackList.tsx

FeedbackModerationPage.tsx

about/

api.ts

types.ts

AboutUsPage.tsx

AboutUsManagementPage.tsx

staff/

api.ts

types.ts

StaffDashboardPage.tsx

StaffOrderListPage.tsx

StaffOrderDetailPage.tsx

PrintInfoPage.tsx

admin/

AdminDashboardPage.tsx

ProductManagementPage.tsx

InventoryPage.tsx

6. State Management Scope

6.1 TanStack Query Usage

TanStack Query chỉ dùng cho các API GET chính.

Data

Hook Example

Current user	useCurrentUserQuery
Product list	useProductsQuery
Product detail	useProductDetailQuery
My designs	useMyDesignsQuery
Try-On status	useTryOnStatusQuery
My orders	useMyOrdersQuery
Order detail	useOrderDetailQuery
Payment status	usePaymentStatusQuery
About Us	useAboutUsQuery
Public feedback	usePublicFeedbackQuery

Không cần setup quá phức tạp. Chỉ cần:

- loading
- error
- refetch
- polling cho Try-On và Payment

6.2 Zustand Usage

Zustand chỉ dùng cho state cần giữ trong client.

Store	Purpose
authStore	accessToken, refreshToken, user
designStore	selected tool, selected color, unsaved changes
checkoutStore	temporary checkout data nếu cần

Không đưa product list, order list, feedback list vào Zustand.

7. API Client Standard

7.1 httpClient Purpose

`httpClient.ts` dùng để:

- Set base URL
- Tự gắn access token
- Bắt lỗi 401
- Trả về data theo format chuẩn
- Hiển thị lỗi dễ hơn cho UI

7.2 API Response Type

```
export type ApiResponse<T> = {  
  success: boolean;  
  message: string;  
  data: T | null;  
  errors: ApiError[] | null;  
  meta: {  
    requestId: string;  
    timestamp: string;  
  };  
};
```

```
export type ApiError = {  
  field?: string;  
  code: string;  
  message: string;  
};
```

7.3 Error Handling UI

Error	Frontend Behavior
400	Hiển thị lỗi dưới field
401	Redirect Login
403	Hiển thị Forbidden Page
404	Hiển thị Not Found
409	Hiển thị toast warning

422	Hiển thị message nghiệp vụ
500	Hiển thị ErrorState + Retry
503	Hiển thị service unavailable

8. Routing Scope

8.1 Public Routes

Path	Page	Role
/	Landing Page	Public
/products	Product Catalog Page	Public
/products/:productId	Product Detail Page	Public
/about-us	About Us Page	Public
/feedbacks	Public Feedback Page	Public
/login	Login Page	Guest
/register	Register Page	Guest

8.2 Customer Routes

Path	Page	Role
/designs/:designId/editor	Design Editor Page	CUSTOMER
/designs/my	My Designs Page	CUSTOMER
/tryon/:designId	Try-On Page	CUSTOMER
/tryon/result/:requestId	Try-On Result Page	CUSTOMER
/checkout/:designId	Checkout Page	CUSTOMER
/payment/:orderId	Payment Page	CUSTOMER

/payment/success	Payment Success Page	CUSTOMER
/payment/cancel	Payment Cancel Page	CUSTOMER
/orders/my	My Orders Page	CUSTOMER
/orders/:orderId	Order Detail Page	CUSTOMER
/orders/:orderId/feedback	Submit Feedback Page	CUSTOMER

8.3 Staff Routes

Path	Page	Role
/staff	Staff Dashboard Page	STAFF, ADMIN
/staff/orders	Staff Order List Page	STAFF, ADMIN
/staff/orders/:orderId	Staff Order Detail Page	STAFF, ADMIN
/staff/orders/:orderId/print	Print Info Page	STAFF, ADMIN
/staff/feedbacks	Feedback Moderation Page	STAFF, ADMIN
/staff/about-us	About Us Management Page	STAFF, ADMIN

8.4 Admin Routes

Path	Page	Role
/admin	Admin Dashboard Page	ADMIN
/admin/products	Product Management Page	ADMIN, STAFF
/admin/inventory	Inventory Page	ADMIN, STAFF
/admin/about-us	About Us Management Page	ADMIN, STAFF

9. Route Guard Rules

9.1 Protected Route

Nếu user chưa đăng nhập mà vào route cần login:

Redirect to /login

9.2 Role Guard

Nếu user đăng nhập rồi nhưng không đúng role:

Show 403 Forbidden

9.3 Redirect After Login

Role	Redirect
CUSTOMER	/
STAFF	/staff
ADMIN	/admin

10. Frontend Pages — Public

10.1 Landing Page

Purpose

Trang giới thiệu nhanh sản phẩm và điều hướng user đến catalog/design flow.

Role

Public

API Used

API

Purpose

GET /api/products	Lấy vài sản phẩm nổi bật
GET /api/feedbacks/public	Lấy feedback công khai
GET /api/about-us	Lấy nội dung giới thiệu ngắn

Components

Component	Purpose
HeroSection	Giới thiệu nền tảng
FeaturedProducts	Hiển thị sản phẩm nổi bật
HowItWorks	Mô tả cách hoạt động
TryOnIntro	Giới thiệu AI Try-On
FeedbackPreview	Hiển thị feedback
CTASection	Kêu gọi xem sản phẩm

Fresher Scope

Chỉ cần UI đơn giản, rõ ràng:

- Không cần animation phức tạp
- Không cần carousel phức tạp
- Có responsive cơ bản
- Có loading skeleton đơn giản

10.2 Product Catalog Page

Purpose

Hiển thị danh sách sản phẩm áo để Guest/Customer chọn.

Role

Public

API Used

API	Purpose
GET /api/products	Lấy danh sách sản phẩm

UI Elements

- Search input
- Product grid
- Product card
- Pagination đơn giản
- Loading state
- Empty state

API Mapping

GET /api/products?keyword=&page=1&pageSize=12

Main Action

Click Product Card
→ Navigate /products/:productId

10.3 Product Detail Page

Purpose

Hiển thị chi tiết sản phẩm, variant và nút bắt đầu thiết kế.

Role

Public

API Used

API	Purpose
GET /api/products/{productId}	Lấy product detail, images, variants, inventory

POST /api/designs

Tạo design draft nếu user bấm Start Design

UI Elements

- Product image
- Product name
- Description
- Price
- Variant selector
- Inventory badge
- Start Design button

Main Actions

Action	Behavior
Select size/color/material	Lưu selected variant trong local state
Click Start Design chưa login	Redirect /login
Click Start Design đã login	Call POST /api/designs
Create design success	Redirect /designs/{designId}/editor

Fresher Scope

Variant selector chỉ cần dropdown hoặc button group đơn giản.

10.4 About Us Page

Purpose

Hiển thị nội dung giới thiệu nền tảng.

Role

Public

API Used

API

Purpose

GET /api/about-us Lấy section published

UI Elements

- About hero
- Section list
- Image nếu có

Matching PRD

PRD Feature

Page

Guest xem About Us

About Us
Page

Customer xem About Us

About Us
Page

10.5 Public Feedback Page

Purpose

Hiện thị feedback đã được duyệt.

Role

Public

API Used

API

Purpose

GET /api/feedbacks/public Lấy feedback APPROVED

UI Elements

- Feedback card
- Rating stars
- Comment
- Feedback image nếu có

11. Frontend Pages — Authentication

11.1 Register Page

Purpose

Cho Guest tạo tài khoản Customer.

API Used

API	Purpose
POST /api/auth/register	Register account

Form Fields

Field	Rule
email	Required
password	Required
fullName	Required
phone	Optional

Success Flow

Register success
→ Show toast
→ Redirect /login

Error Handling

Error Code	UI
EMAIL_ALREADY_EXISTS	Show under email

PASSWORD_WEAK	Show under password
INTERNAL_SERVER_ERROR	Show toast

11.2 Login Page

Purpose

Cho user đăng nhập vào hệ thống.

API Used

API	Purpose
POST /api/auth/login	Login
GET /api/auth/me	Get current user

Success Flow

Login success
→ Save accessToken, refreshToken
→ Save user info
→ Redirect by role

Redirect Rule

Role	Redirect
CUSTOMER	/
STAFF	/staff
ADMIN	/admin

12. Frontend Pages — Design

12.1 My Designs Page

Purpose

Cho Customer xem danh sách thiết kế của mình.

API Used

API	Purpose
GET /api/designs/my	Lấy design của customer

UI Elements

- Design grid
- Design card
- Status badge
- Edit button
- Try-On button
- Checkout button

Action Rules

Status	Actions
DRAFT	Continue Edit
SAVED	Edit, Try-On, Checkout
LOCKED	View only

12.2 Design Editor Page

Purpose

Cho Customer thiết kế áo bằng Fabric.js.

Role

CUSTOMER

API Used

API	Purpose
GET /api/designs/{designId}	Load design
PUT /api/designs/{designId}/save	Save design

Fabric.js MVP Features

Frontend chỉ cần làm các tính năng cơ bản:

Feature	Required
Add text	Yes
Edit text	Yes
Change text color	Yes
Upload image	Yes
Move object	Yes
Resize object	Yes
Rotate object	Yes
Delete object	Yes
Export canvas JSON	Yes
Export preview image	Yes
Layer panel nâng cao	No
Multi-select	No
Snap grid	No
Advanced typography	No
Template system	No

UI Layout

Area	Purpose
------	---------

Left toolbar	Add text, upload image, delete
Center canvas	T-shirt design area
Right panel	Object properties
Top bar	Back, Save, Try-On, Checkout

Components

Component	Purpose
DesignEditorPage	Main page
TShirtCanvas	Fabric canvas
EditorToolbar	Add text/image/delete
ObjectPropertyPane	Edit selected object
SaveDesignButton	Save API
DesignPreview	Show product/design preview

Save Design Flow

User clicks Save
 → Export canvas JSON
 → Export preview image URL or base64 upload result
 → Build layer array
 → PUT /api/designs/{designId}/save
 → Show success toast

Request Mapping

Frontend gửi:

name
 canvasJson
 previewImageUrl
 printFileUrl
 layers

Important Rules

Rule	UI Behavior
Design LOCKED	Disable editing
Save loading	Disable Save button
Save success	Show toast
Save failed	Show error
Leave page with unsaved changes	Show confirm modal

Fresher Scope Note

Không cần làm editor quá chuyên nghiệp. Chỉ cần:

Text + image + move + resize + rotate + save

là đủ cho MVP.

13. Frontend Pages — AI Try-On

13.1 Try-On Page

Purpose

Cho Customer upload ảnh và gửi yêu cầu AI Try-On.

API Used

API	Purpose
GET /api/designs/{designId}	Lấy design
POST /api/tryon/requests	Tạo Try-On request

UI Elements

- Design preview
- Upload user photo
- Height input
- Weight input
- Submit Try-On button

Success Flow

Create Try-On request success

→ Redirect /tryon/result/{requestId}

13.2 Try-On Result Page

Purpose

Hiển thị trạng thái xử lý AI và kết quả Try-On.

API Used

API	Purpose
GET /api/tryon/requests/{requestId}	Check status
GET /api/tryon/requests/{requestId}/result	Get result

Polling Rule

Dùng TanStack Query polling đơn giản:

Poll every 3 seconds

Stop when status = SUCCEEDED or FAILED

UI By Status

Status	UI
PENDING	Waiting message
PROCESSING	Processing loading

SUCCEEDED	Show result image
FAILED	Show error and retry button

Important Rule

Try-On image chỉ để preview.

Không dùng Try-On result image làm file in

14. Frontend Pages — Checkout & Payment

14.1 Checkout Page

Purpose

Cho Customer xác nhận sản phẩm, design, địa chỉ giao hàng và tạo order.

API Used

API	Purpose
GET /api/designs/{designId}	Lấy design
GET /api/products/{productId}	Lấy product/variant
POST /api/orders	Tạo order

Form Fields

Field	Rule
receiverName	Required
receiverPhone	Required
shippingAddress	Required

quantity Required, > 0

UI Elements

- Design preview
- Product summary
- Variant summary
- Quantity input
- Shipping form
- Price summary
- Create order button

Success Flow

Create order success
→ Redirect /payment/{orderId}

14.2 Payment Page

Purpose

Cho Customer thanh toán order bằng PayOS hoặc SePay.

API Used

API	Purpose
POST /api/payments	Tạo payment
GET /api/payments/order/{orderId}	Kiểm tra payment status

Provider Options

Provider	Frontend UI
PAYOS	Show checkout URL button
SEPAY	Show QR code + bank info + transfer content

UI Elements

- Payment method selector
- PayOS payment box
- SePay QR payment box
- Payment instruction
- Payment status box

PayOS Flow

User chooses PAYOS
 → POST /api/payments
 → Receive checkoutUrl
 → Click Pay Now
 → Redirect to PayOS
 → Return to /payment/success

SePay Flow

User chooses SEPAY
 → POST /api/payments
 → Show QR + bank info
 → User transfers money
 → FE polls payment status
 → If PAID redirect /payment/success

Polling Rule

GET /api/payments/order/{orderId}
 every 5 seconds while paymentStatus = PENDING

UI By Payment Status

Status	UI
PENDING	Show payment instruction
PAID	Redirect success
FAILED	Show retry payment
CANCELLED	Show cancel message

Fresher Scope

Không cần tự xử lý webhook trên frontend. Webhook là backend C# xử lý.

Frontend chỉ cần:

Create payment

Show payment info

Poll payment status

Show success/cancel

14.3 Payment Success Page

Purpose

Hiển thị thanh toán thành công.

API Used

API	Purpose
GET /api/payments/order/{orderId}	Lấy payment
GET /api/orders/{orderId}	Lấy order
GET /api/payments/{paymentId}/invoice	Lấy invoice PDF

UI Elements

- Success icon
- Order code
- Payment amount
- Invoice download button
- Go to My Orders button

14.4 Payment Cancel Page

Purpose

Hiển thị thanh toán bị hủy hoặc chưa hoàn tất.

UI Actions

Action	Behavior
--------	----------

Retry payment	Go back Payment Page
View order	Go Order Detail
Back home	Go Landing

15. Frontend Pages — Orders

15.1 My Orders Page

Purpose

Cho Customer xem danh sách đơn hàng của mình.

API Used

API	Purpose
GET /api/orders/my	Lấy order list

UI Elements

- Order card
 - Order code
 - Total amount
 - Payment status badge
 - Order status badge
 - View detail button
-

15.2 Order Detail Page

Purpose

Cho Customer xem chi tiết đơn hàng, trạng thái, payment và invoice.

API Used

API	Purpose
GET /api/orders/{orderId}	Lấy chi tiết order
GET /api/payments/order/{orderId}	Lấy payment
GET /api/payments/{paymentId}/invoice	Lấy invoice

UI Elements

- Order summary
- Order item list
- Order timeline
- Shipping info
- Payment info
- Invoice download button
- Feedback button

Feedback Button Rule

Chỉ hiện nút feedback khi:

orderStatus = COMPLETED

16. Frontend Pages — Feedback

16.1 Submit Feedback Page

Purpose

Cho Customer gửi feedback sau khi đơn hoàn tất.

API Used

API	Purpose
POST /api/feedbacks	Submit feedback

Form Fields

Field	Rule
rating	Required, 1-5
comment	Optional
imageUrl	Optional

Success Flow

Submit success
→ Show pending moderation message
→ Redirect order detail

16.2 Public Feedback Component

Purpose

Hiển thị feedback public ở Landing hoặc Product Detail.

API Used

API	Purpose
GET /api/feedbacks/public	Get approved feedback

16.3 Feedback Moderation Page

Purpose

Cho Staff/Admin duyệt feedback.

API Used

API	Purpose
PATCH /api/admin/feedbacks/{feedbackId}/status	Update feedback status

Actions

Action	Status
Approve	APPROVED
Hide	HIDDEN
Reject	REJECTED

Fresher Scope

Có thể làm list feedback đơn giản. Không cần dashboard moderation phức tạp.

17. Frontend Pages — Staff

17.1 Staff Dashboard Page

Purpose

Trang tổng quan đơn giản cho Staff.

API Used

API	Purpose
GET /api/staff/orders	Lấy danh sách order cần xử lý

UI Elements

- Total paid orders
- Total in production orders
- Recent order list
- Link to Staff Orders

Fresher Scope

Dashboard chỉ cần card số liệu đơn giản, không cần chart.

17.2 Staff Order List Page

Purpose

Cho Staff xem đơn đã thanh toán hoặc đang xử lý.

API Used

API	Purpose
GET /api/staff/orders	Lấy order list

Filters

Filter	Values
status	PAID, IN_PRODUCTION, SHIPPING, COMPLETED
keyword	orderCode

UI Elements

- Order table
 - Status badge
 - View detail button
-

17.3 Staff Order Detail Page

Purpose

Cho Staff xem chi tiết đơn và cập nhật trạng thái.

API Used

API	Purpose
GET /api/orders/{orderId}	Lấy order detail

PATCH /api/staff/orders/{orderId}/status	Update order status
--	---------------------

UI Elements

- Order info
- Customer shipping info
- Order items
- Status update buttons
- Print info button

Allowed Actions

Current Status	Action
PAID	Start Production
IN_PRODUCTION	Mark Shipping
SHIPPING	Mark Completed

17.4 Print Info Page

Purpose

Cho Staff xem file thiết kế để in.

API Used

API	Purpose
GET /api/staff/orders/{orderId}/print-info	Lấy print info

UI Elements

- Product snapshot
- Variant info
- Quantity
- Design preview
- Print file download button

Important Rule

Staff chỉ dùng:

`design.printFileUrl`

Không dùng:

`tryonResult.resultImageUrl`

18. Frontend Pages — Admin

18.1 Admin Dashboard Page

Purpose

Trang tổng quan đơn giản cho Admin.

Fresher Scope

Chỉ cần link nhanh đến:

- Product Management
- Inventory
- Orders
- Feedback
- About Us

Không cần biểu đồ nâng cao.

18.2 Product Management Page

Purpose

Cho Admin/Staff tạo sản phẩm mới.

API Used

API	Purpose
GET /api/products	Lấy product list
POST /api/admin/products	Tạo product

Form Scope

Product form gồm:

- name
- description
- basePrice
- imageUrl
- variant list đơn giản
- initial inventory

Fresher Scope

Không cần edit/delete product trong phase đầu. Chỉ cần:

List product + Create product

18.3 Inventory Page

Purpose

Cho Admin/Staff cập nhật tồn kho.

API Used

API	Purpose
GET /api/products	Lấy product
GET /api/products/{productId}	Lấy variant
PATCH /api/admin/product-variants/{variantId}/inventory	Update inventory

UI Elements

- Product selector
- Variant list

- Quantity input
 - Update button
-

18.4 About Us Management Page

Purpose

Cho Staff/Admin cập nhật nội dung About Us.

API Used

API	Purpose
GET /api/about-us	Lấy content
PUT /api/admin/about-us/{sectionKey}	Update content

UI Elements

- Section selector
 - Title input
 - Content textarea
 - Image URL input
 - Status selector
 - Save button
-

19. Component Scope

19.1 Shared Components

Frontend fresher chỉ cần các component cơ bản:

Component	Purpose
Button	Button dùng chung
Input	Input dùng chung
Textarea	Textarea dùng chung

Select	Dropdown
Card	Box nội dung
Badge	Hiển thị status
Modal	Confirm đơn giản
Loading	Loading spinner
EmptyState	Không có dữ liệu
ErrorState	Lỗi API
ProtectedRoute	Chặn route chưa login
RoleGuard	Chặn route sai role

19.2 Domain Components

Domain	Components
Catalog	ProductCard, ProductGrid, VariantSelector
Design	TShirtCanvas, EditorToolbar, SaveDesignButton
Try-On	TryOnUploadForm, TryOnStatusBox
Order	OrderCard, OrderStatusBadge, OrderTimeline
Payment	PayOSBox, SePayBox, PaymentStatusBadge
Staff	StaffOrderTable, PrintInfoCard
Feedback	RatingInput, FeedbackCard
About	AboutSectionBlock, AboutEditor

20. UI State Rules

20.1 Loading State

Page	Loading UI
------	------------

Product list	Product card skeleton
Product detail	Basic loading
Design detail	Loading editor
Try-On	Processing message
Payment	Payment loading
Orders	Order skeleton

20.2 Empty State

Page	Empty Message
Product list	No products found
My designs	You have no designs yet
My orders	You have no orders yet
Feedback	No feedback yet
Staff orders	No orders to process

20.3 Error State

Error	UI
Network error	Show retry button
401	Redirect login
403	Forbidden page
404	Not found page
422	Show business message
500	Show generic error

21. Frontend Validation Rules

21.1 Register Form

Field	Rule
email	Required, email format
password	Required, min 8 characters
fullName	Required
phone	Optional

21.2 Product Form

Field	Rule
name	Required
basePrice	Required, ≥ 0
imageUrl	Required
sku	Required
size	Required
color	Required
material	Required
availableQuantity	Required, ≥ 0

21.3 Design Save

Field	Rule
name	Required
canvasJson	Required
previewImageUrl	Required
printFileUrl	Required

layers	Can be empty but should be valid array
--------	--

21.4 Checkout Form

Field	Rule
receiverName	Required
receiverPhone	Required
shippingAddress	Required
quantity	Required, > 0

21.5 Feedback Form

Field	Rule
rating	Required, 1-5
comment	Optional
imageUrl	Optional

22. FE Team Split — 2 Fresher Developers

22.1 Frontend Developer 1

Main responsibility:

Public + Auth + Product + Admin Product

Tasks

Area	Task
Setup	Project setup, Tailwind, router

Shared	Button, Input, Card, Badge, Loading
Auth	Login, Register, authStore
Public	Landing, About Us, Feedback public
Catalog	Product list, product detail
Admin	Product create page
Inventory	Inventory update page

APIs Owned

API

POST /api/auth/register

POST /api/auth/login

GET /api/auth/me

GET /api/products

GET /api/products/{productId}

POST /api/admin/products

PATCH /api/admin/product-variants/{variantId}/inventory

GET /api/about-us

PUT /api/admin/about-us/{sectionKey}

GET /api/feedbacks/public

22.2 Frontend Developer 2

Main responsibility:

Design + Try-On + Order + Payment + Staff

Tasks

Area	Task
Design	My Designs, Design Editor basic
Fabric.js	Add text, upload image, save canvas

Try-On	Upload photo, request Try-On, result polling
Checkout	Checkout form, create order
Payment	PayOS/SePay UI, payment polling
Orders	My Orders, Order Detail
Staff	Staff order list, detail, print info
Feedback	Submit feedback, moderate feedback
k	

APIs Owned

API

POST /api/designs

PUT /api/designs/{designId}/save

GET /api/designs/my

GET /api/designs/{designId}

POST /api/tryon/requests

GET /api/tryon/requests/{requestId}

GET /api/tryon/requests/{requestId}/result

POST /api/orders

GET /api/orders/my

GET /api/orders/{orderId}

PATCH /api/staff/orders/{orderId}/status

POST /api/payments

GET /api/payments/order/{orderId}

GET /api/payments/{paymentId}/invoice

POST /api/feedbacks

PATCH /api/admin/feedbacks/{feedbackId}/status

GET /api/staff/orders

GET /api/staff/orders/{orderId}/print-info

23. Development Phases

Phase 1 — Frontend Foundation

Goal:

App chạy được, routing được, gọi API được

Tasks:

- Setup React + TypeScript + Tailwind
 - Setup React Router
 - Setup Axios httpClient
 - Setup TanStack Query provider
 - Setup authStore
 - Setup shared components
 - Setup ProtectedRoute
 - Setup RoleGuard
-

Phase 2 — Public + Auth

Goal:

Guest xem được web và login/register được

Tasks:

- Landing Page
 - Product Catalog
 - Product Detail
 - About Us
 - Public Feedback
 - Register Page
 - Login Page
 - Redirect by role
-

Phase 3 — Customer Design

Goal:

Customer tạo và lưu thiết kế được

Tasks:

- Create design from product detail
 - My Designs Page
 - Design Editor Page
 - Fabric.js canvas
 - Add text
 - Upload image
 - Move/resize/rotate object
 - Save design API
-

Phase 4 — Try-On

Goal:

Customer gửi Try-On và xem result được

Tasks:

- Try-On upload form
 - Request Try-On API
 - Try-On status polling
 - Try-On result page
 - Failed/retry UI
-

Phase 5 — Checkout + Payment

Goal:

Customer tạo order và thanh toán được

Tasks:

- Checkout Page
- Create Order API
- Payment Page
- PayOS UI
- SePay QR UI
- Payment status polling

- Payment Success Page
 - Invoice download button
-

Phase 6 — Staff/Admin Basic

Goal:

Staff/Admin xử lý demo vận hành được

Tasks:

- Staff Dashboard
 - Staff Order List
 - Staff Order Detail
 - Print Info Page
 - Update Order Status
 - Feedback Moderation
 - Product Management basic
 - Inventory Management basic
 - About Us Management basic
-

24. PRD Traceability Matrix

PRD Feature	Frontend Page	API
Register	Register Page	POST /api/auth/register
Login	Login Page	POST /api/auth/login
View About Us	About Us Page	GET /api/about-us
Manage About Us	About Us Management	PUT /api/admin/about-us/{sectionKey}
View Product	Product Catalog	GET /api/products
Product Detail	Product Detail Page	GET /api/products/{productId}
Select Variant	Product Detail Page	GET /api/products/{productId}

Create Design	Product Detail / Editor	POST /api/designs
Save Design	Design Editor	PUT /api/designs/{designId}/save
My Designs	My Designs Page	GET /api/designs/my
AI Try-On	Try-On Page	POST /api/tryon/requests
Try-On Status	Try-On Result Page	GET /api/tryon/requests/{requestId}
Try-On Result	Try-On Result Page	GET /api/tryon/requests/{requestId}/result
Create Order	Checkout Page	POST /api/orders
Real Payment	Payment Page	POST /api/payments
Payment Status	Payment Page	GET /api/payments/order/{orderId}
Invoice PDF	Order Detail	GET /api/payments/{paymentId}/invoice
My Orders	My Orders Page	GET /api/orders/my
Order Detail	Order Detail Page	GET /api/orders/{orderId}
Staff Update Order	Staff Order Detail	PATCH /api/staff/orders/{orderId}/status
Print Info	Print Info Page	GET /api/staff/orders/{orderId}/print-info
Submit Feedback	Submit Feedback Page	POST /api/feedbacks
Public Feedback	Landing/Product Detail	GET /api/feedbacks/public
Moderate Feedback	Feedback Moderation	PATCH /api/admin/feedbacks/{feedbackId}/status
Product Management	Product Management	POST /api/admin/products
Inventory Management	Inventory Page	PATCH /api/admin/product-variants/{variantId}/inventory

25. Definition of Done

Frontend MVP được xem là hoàn thành khi:

Public

- Guest xem được Landing Page
- Guest xem được Product Catalog
- Guest xem được Product Detail
- Guest xem được About Us
- Guest xem được Public Feedback

Auth

- User register được
- User login được
- Token được lưu
- Route guard hoạt động
- Redirect theo role hoạt động

Customer

- Customer tạo design draft được
- Customer mở Design Editor được
- Customer thêm text/image vào áo được
- Customer save design được
- Customer gửi Try-On request được
- Customer xem Try-On result được
- Customer checkout được
- Customer tạo payment PayOS/SePay được
- Customer xem payment status được
- Customer xem order detail được
- Customer tải invoice PDF được
- Customer gửi feedback được khi order completed

Staff/Admin

- Staff xem order list được
- Staff xem order detail được
- Staff xem print info được
- Staff update order status được
- Staff/Admin duyệt feedback được
- Staff/Admin tạo product cơ bản được
- Staff/Admin cập nhật inventory được
- Staff/Admin cập nhật About Us được

UX

- Có loading state
 - Có empty state
 - Có error state
 - Có toast success/error
 - UI responsive cơ bản
 - Không crash khi API lỗi
-

26. Final Frontend Decision

Frontend MVP dành cho **2 fresher frontend developers** nên giữ scope vừa đủ.

Final stack:

React + TypeScript + Tailwind CSS + Axios + React Router + TanStack Query basic + Zustand minimal + Fabric.js basic

Frontend không cần làm quá phức tạp.

Mục tiêu chính:

Làm được flow end-to-end

Gọi API đúng

UI rõ ràng

Role guard đúng

Design Editor cơ bản chạy được

Payment PayOS/SePay hiển thị được

Staff xử lý đơn được

PRD Lite, API doc và DB 18 bảng.

Diagram-Contain Diagram

@startuml

title AI T-Shirt Customization Platform - Container Diagram

skinparam backgroundColor white
skinparam shadowing false
skinparam roundcorner 15
skinparam packageStyle rectangle

actor "Customer" as Customer
actor "Staff / Admin" as Staff

rectangle "AI T-Shirt Platform\nSystem Boundary" as System {

 rectangle "Web Frontend\nReact + TypeScript\nTailwind CSS\nFabric.js Basic\n2 FE Fresher" as FE #E0F2FE

 rectangle "API Entry Layer\nREST API\nJWT Protected APIs\nRole Guard" as API #F8FAFC

 package "C# Platform Services\nFresher Scope" {

 rectangle "Identity Service\nC# .NET 8\n\nRegister/Login\nJWT\nRefresh Token\nRBAC\n\nOwns:\nusers,\nroles,\nuser_roles,\nrefresh_tokens" as Identity #FEF3C7

 rectangle "Payment & Invoice Service\nC# .NET 8\n\nPayOS / SePay\nWebhook\nPayment Status\nInvoice PDF\n\nOwns:\npayments" as Payment #FEF3C7

 rectangle "Staff Operation Gateway\nC# .NET 8\n\nValidate Staff/Admin\nPrint Info\nAbout Us\nCall Java APIs\n\nOwns:\nabout_us_contents" as StaffGateway #FEF3C7
 }

 package "Java Core Domain Services\nJunior Core Scope" {

 rectangle "Catalog Service\nJava Spring Boot\n\nProduct\nVariant\nImage\nInventory\n\nOwns:\nproducts,\nproduct_variants,\nproduct_images,\nproduct_inventory" as Catalog #DCFCE7

 rectangle "Design Try-On Service\nJava Spring Boot\n\nSave Design\nSave Layers\nTry-On Request\nTry-On Result\nLock Design\n\nOwns:\ndesigns,\ndesign_layers,\ntryon_requests,\ntryon_results" as Design #DCFCE7

 rectangle "Order Service\nJava Spring Boot\n\nCreate Order\nOrder Items\nStatus Lifecycle\nConsume PaymentSucceeded\n\nOwns:\norders,\norder_items,\norder_status_history" as Order #DCFCE7

 rectangle "Feedback Service\nJava Spring Boot\n\nSubmit Feedback\nModerate Feedback\nPublic Feedback\n\nOwns:\nfeedbacks" as Feedback #DCFCE7
 }

 queue "Kafka Event Bus\n\nOrderCreated\nPaymentSucceeded\nPaymentFailed\nTryOnRequested\nTryOnCompleted\nOrderCompleted" as Kafka #F3E8FF

 rectangle "AI Try-On Worker\n\nMock AI in MVP\nConsume TryOnRequested\nProduce TryOnCompleted" as AI #E0E7FF
}

database "PostgreSQL Database\n18 Tables\n\nidentity: users, roles, user_roles, refresh_tokens\ncatalog: products, variants, images, inventory\ndesign: designs, design_layers\nai_tryon: requests, results\nordering: orders, items, history\npayment: payments\nfeedback: feedbacks\ncontent: about_us_contents" as DB #E5E7EB

database "Object Storage\n\nProduct images\nDesign preview\nPrint files\nTry-On results\nInvoice PDFs" as Storage #E5E7EB

cloud "PayOS / SePay\nExternal Payment Provider\n\nCheckout link\nQR payment\nBank transfer\nWebhook callback" as Provider #FFE4E6

Customer --> FE : Uses web app

Staff --> FE : Uses staff/admin UI

FE --> API : REST API + JWT

API --> Identity : Auth APIs

API --> Payment : Payment APIs

API --> StaffGateway : Staff/Admin APIs

API --> Catalog : Product APIs

API --> Design : Design / Try-On APIs

API --> Order : Order APIs

API --> Feedback : Feedback APIs

Identity --> DB : Read/Write identity tables

Payment --> DB : Read/Write payments

StaffGateway --> DB : Read/Write content

Catalog --> DB : Read/Write catalog tables

Design --> DB : Read/Write design + tryon tables

Order --> DB : Read/Write order tables

Feedback --> DB : Read/Write feedbacks

Catalog --> Storage : Product images

Design --> Storage : Design preview + print file

AI --> Storage : Try-On result image

Payment --> Storage : Invoice PDF

Payment --> Provider : Create payment link / QR

Provider -[#red,dashed]-> Payment : Payment webhook

Order ..> Kafka : Publish OrderCreated

Kafka ..> Payment : Consume OrderCreated

Payment ..> Kafka : Publish PaymentSucceeded

Kafka ..> Order : Consume PaymentSucceeded

Design ..> Kafka : Publish TryOnRequested

Kafka ..> AI : Consume TryOnRequested

AI ..> Kafka : Publish TryOnCompleted

Kafka ..> Design : Consume TryOnCompleted

Order ..> Kafka : Publish OrderCompleted

Kafka ..> Feedback : Enable feedback flow

StaffGateway --> Order : Update order status

StaffGateway --> Design : Get print file info

StaffGateway --> Feedback : Moderate feedback

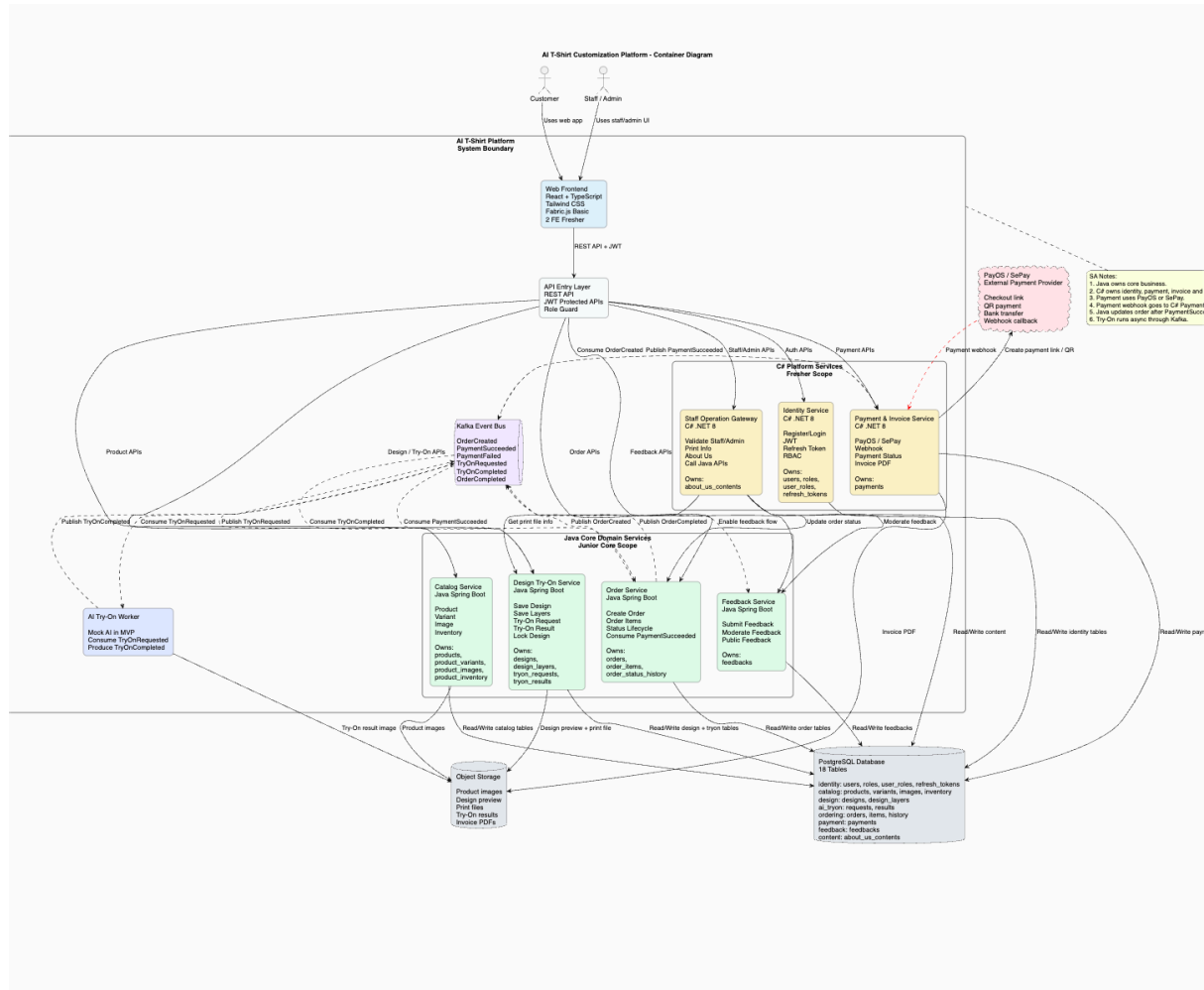
note bottom of System

SA Notes:

1. Java owns core business.

2. C# owns identity, payment, invoice and staff gateway.
 3. Payment uses PayOS or SePay.
 4. Payment webhook goes to C# Payment Service.
 5. Java updates order after PaymentSucceeded event.
 6. Try-On runs async through Kafka.
- end note

@enduml



Web Flow Diagram

@startuml

title AI T-Shirt Customization Platform\nSimplified System Flow Diagram

skinparam backgroundColor white
skinparam shadowing false
skinparam roundcorner 15
skinparam packageStyle rectangle

actor "User\nCustomer / Staff / Admin" as User

rectangle "Web Frontend\nReact + TypeScript\nTailwind CSS\nFabric.js Basic" as FE #E0F2FE

rectangle "API Gateway / API Entry Layer\nREST API\nJWT Authentication\nRole Guard" as API #F8FAFC

queue "Kafka Event Bus\n\nOrderCreated\nPaymentSucceeded\nTryOnRequested\nTryOnCompleted" as Kafka #F3E8FF

rectangle "Backend Java System\nSpring Boot\n\nCore Business Domain\n\nCatalog Service\nDesign Service\nTry-On Service\nOrder Service\nFeedback Service" as JavaSystem #DCFCE7

rectangle "Backend C# System\n.NET 8\n\nPlatform / Integration Domain\n\nIdentity Service\nPayment Service\nInvoice Service\nStaff Gateway\nAbout Us Service" as CSharpSystem #FEF3C7

database "PostgreSQL Database\n18 Tables\n\nusers / roles\nproducts / variants\ndesigns / layers\ntryon requests / results\norders / order items\npayments\nfeedbacks\nabout us" as DB #E5E7EB

cloud "PayOS / SePay\nExternal Payment Provider" as PaymentProvider #FFE4E6

User --> FE : Uses web app

FE --> API : Calls REST API

API --> Kafka : Publish / route events

Kafka --> JavaSystem : Event to Java backend

Kafka --> CSharpSystem : Event to C# backend

JavaSystem --> DB : Read / Write core business data

CSharpSystem --> DB : Read / Write auth, payment, content data

CSharpSystem --> PaymentProvider : Create payment link / QR

PaymentProvider -[#red,dashed]-> CSharpSystem : Payment webhook

note bottom of JavaSystem

Java System handles:

Product, Design, Try-On, Order, Feedback.

end note

note bottom of CSharpSystem

C# System handles:

Auth, Payment, Invoice, Staff Gateway, About Us.

end note

note bottom of DB

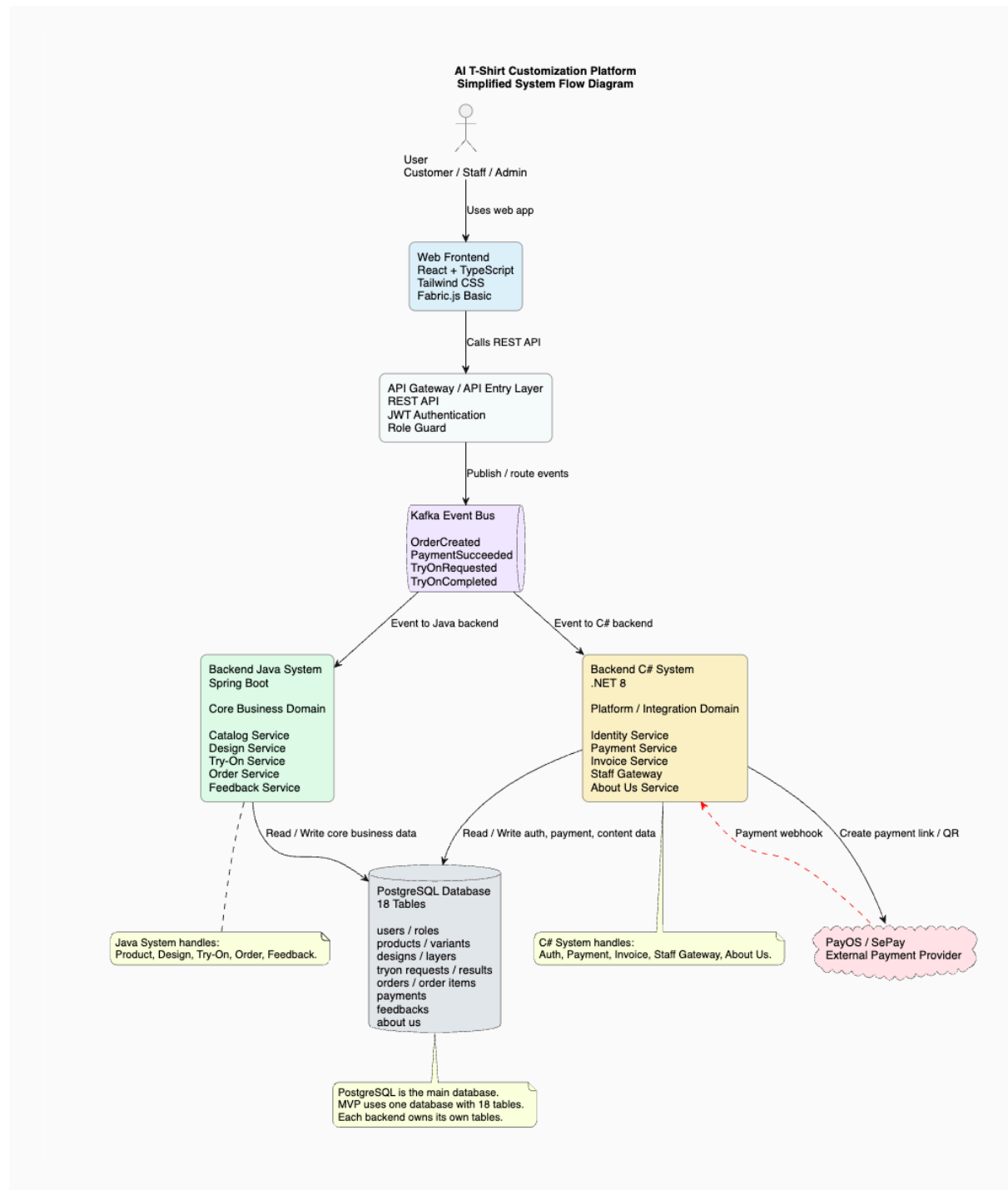
PostgreSQL is the main database.

MVP uses one database with 18 tables.

Each backend owns its own tables.

end note

@enduml



AI Try-On Flow

@startuml

title Sequence Diagram - AI Try-On Flow

actor "Customer" as Customer
participant "Web Frontend" as FE
participant "API Entry Layer" as API
participant "Java Design Try-On Service" as TryOn
participant "Kafka Event Bus" as Kafka
participant "AI Try-On Worker" as AI
database "PostgreSQL Database" as DB
database "Object Storage" as Storage

Customer -> FE : Open Try-On page
FE -> API : GET /api/designs/{designId}
API -> TryOn : Get design detail
TryOn -> DB : Read design and layers
DB --> TryOn : Design data
TryOn --> API : Return design detail
API --> FE : Show design preview

Customer -> FE : Upload user photo
FE -> Storage : Upload photo
Storage --> FE : userPhotoUrl

Customer -> FE : Click Try-On
FE -> API : POST /api/tryon/requests
API -> TryOn : Create Try-On request

TryOn -> DB : Insert tryon_request status = PENDING
TryOn -> Kafka : Publish TryOnRequested
TryOn --> API : Return tryonRequestId
API --> FE : 202 Accepted

loop Frontend polling every 3 seconds
FE -> API : GET /api/tryon/requests/{requestId}
API -> TryOn : Get Try-On status
TryOn -> DB : Read tryon_request
DB --> TryOn : PENDING / PROCESSING / SUCCEEDED
TryOn --> API : Return status
API --> FE : Show current status
end

Kafka -> AI : Consume TryOnRequested

AI -> DB : Update request status = PROCESSING
AI -> Storage : Read user photo and design preview
Storage --> AI : Input images

AI -> AI : Process Try-On image
AI -> Storage : Save result image
Storage --> AI : resultImageUrl

AI -> Kafka : Publish TryOnCompleted

Kafka -> TryOn : Consume TryOnCompleted
TryOn -> DB : Update tryon_request status = SUCCEEDED
TryOn -> DB : Insert tryon_result
DB --> TryOn : Saved

FE -> API : GET /api/tryon/requests/{requestId}/result
API -> TryOn : Get Try-On result
TryOn -> DB : Read tryon_result
DB --> TryOn : resultImageUrl
TryOn --> API : Return Try-On result
API --> FE : Show Try-On result image

note right of TryOn
Kafka is used because AI Try-On is long-running.
The user does not wait for AI inside one HTTP request.
end note

note right of FE
Try-On result is only for preview.
Print file still comes from saved design.
end note

@enduml

Order Payment Flow

@startuml

title Sequence Diagram - Order and Payment Flow

actor "Customer" as Customer

participant "Web Frontend" as FE

participant "API Entry Layer" as API

participant "Java Order Service" as Order

participant "Java Design Service" as Design

participant "C# Payment Service" as Payment

participant "Kafka Event Bus" as Kafka

participant "PayOS / SePay" as Provider

database "PostgreSQL Database" as DB

database "Object Storage" as Storage

Customer -> FE : Open Checkout page

FE -> API : GET /api/designs/{designId}

API -> Design : Get design detail

Design -> DB : Read design

DB --> Design : Design data

Design --> API : Return design detail

API --> FE : Show checkout information

Customer -> FE : Fill shipping info and click Create Order

FE -> API : POST /api/orders

API -> Order : Create order

Order -> DB : Check product, variant and inventory

DB --> Order : Valid product and inventory

Order -> DB : Check design owner and design status

DB --> Order : Design is SAVED

Order -> DB : Insert order status = PENDING_PAYMENT

Order -> DB : Insert order_items

Order -> DB : Insert order_status_history

Order -> Kafka : Publish OrderCreated

Order --> API : Return orderId and totalAmount

API --> FE : Redirect to Payment page

note right of Order

REST is used here because frontend needs orderId immediately.

Kafka event is published after order is saved.

end note

Customer -> FE : Choose PayOS or SePay

FE -> API : POST /api/payments

API -> Payment : Create payment

Payment -> Order : Internal REST API: Get order payment info
Order -> DB : Read order total amount and status
DB --> Order : Order payment info
Order --> Payment : Return order payment info

Payment -> DB : Insert payment status = PENDING

alt PayOS selected

Payment -> Provider : Create payment link

Provider --> Payment : checkoutUrl and qrCodeUrl

else SePay selected

Payment -> Provider : Create QR or bank transfer instruction

Provider --> Payment : qrCodeUrl and bank info

end

Payment -> DB : Update payment provider data

Payment --> API : Return payment info

API --> FE : Show payment instruction

alt PayOS payment

Customer -> FE : Click Pay Now

FE -> Provider : Redirect to checkoutUrl

Customer -> Provider : Complete payment

else SePay payment

Customer -> Provider : Bank transfer with payment content

end

Provider -> Payment : Payment webhook

Payment -> Payment : Verify webhook signature or auth

Payment -> DB : Find payment by provider id or payment content

DB --> Payment : Payment record

Payment -> Payment : Validate amount and status

Payment -> DB : Update payment status = PAID

Payment -> Storage : Generate and save invoice PDF

Storage --> Payment : invoicePdfUrl

Payment -> DB : Save invoice info

Payment -> Kafka : Publish PaymentSucceeded

Kafka -> Order : Consume PaymentSucceeded

Order -> DB : Update order paymentStatus = PAID

Order -> DB : Update orderStatus = PAID

Order -> DB : Lock design

Order -> DB : Insert order_status_history

loop Frontend polling payment status
FE -> API : GET /api/payments/order/{orderId}
API -> Payment : Get payment by order
Payment -> DB : Read payment
DB --> Payment : paymentStatus = PAID
Payment --> API : Return payment status
API --> FE : Show payment success
end

FE -> API : GET /api/orders/{orderId}
API -> Order : Get order detail
Order -> DB : Read order detail
DB --> Order : Order data
Order --> API : Return order detail
API --> FE : Show order detail

note right of Payment
C# owns payment and invoice.
Provider webhook returns to C# Payment Service.
end note

note right of Order
Java owns order.
Java updates order only after PaymentSucceeded event.
end note

@enduml

Staff Order Processing Flow

@startuml

title Sequence Diagram - Staff Order Processing Flow

actor "Staff / Admin" as Staff
participant "Web Frontend" as FE
participant "API Entry Layer" as API
participant "C# Staff Operation Gateway" as StaffGateway
participant "Java Order Service" as Order
participant "Java Design Service" as Design
participant "Kafka Event Bus" as Kafka
database "PostgreSQL Database" as DB
database "Object Storage" as Storage

Staff -> FE : Open Staff Dashboard
FE -> API : GET /api/staff/orders?status=PAID
API -> StaffGateway : Validate Staff/Admin role

StaffGateway -> Order : REST get paid orders
Order -> DB : Read orders where status = PAID
DB --> Order : Paid order list
Order --> StaffGateway : Return order list
StaffGateway --> API : Return staff order list
API --> FE : Show paid orders

Staff -> FE : Open order detail
FE -> API : GET /api/staff/orders/{orderId}
API -> StaffGateway : Validate Staff/Admin role

StaffGateway -> Order : REST get order detail
Order -> DB : Read order, order_items and status_history
DB --> Order : Order detail
Order --> StaffGateway : Return order detail
StaffGateway --> API : Return order detail
API --> FE : Show order detail

Staff -> FE : Open print info
FE -> API : GET /api/staff/orders/{orderId}/print-info
API -> StaffGateway : Validate Staff/Admin role

StaffGateway -> Order : REST get order items
Order -> DB : Read order_items
DB --> Order : Items with designId
Order --> StaffGateway : Return order items

StaffGateway -> Design : REST get design print info
Design -> DB : Read design
DB --> Design : previewImageUrl and printFileUrl
Design -> Storage : Check print file exists

Storage --> Design : File available
Design --> StaffGateway : Return print info

StaffGateway --> API : Return print package
API --> FE : Show print info

Staff -> FE : Click Start Production
FE -> API : PATCH /api/staff/orders/{orderId}/status
API -> StaffGateway : Validate Staff/Admin role

StaffGateway -> Order : REST update status to IN_PRODUCTION
Order -> DB : Check current status = PAID
DB --> Order : Valid current status
Order -> DB : Update orderStatus = IN_PRODUCTION
Order -> DB : Insert order_status_history
Order --> StaffGateway : Return success
StaffGateway --> API : Return success
API --> FE : Show IN_PRODUCTION

Staff -> FE : Click Mark Shipping
FE -> API : PATCH /api/staff/orders/{orderId}/status
API -> StaffGateway : Validate Staff/Admin role

StaffGateway -> Order : REST update status to SHIPPING
Order -> DB : Check current status = IN_PRODUCTION
DB --> Order : Valid current status
Order -> DB : Update orderStatus = SHIPPING
Order -> DB : Insert order_status_history
Order --> StaffGateway : Return success
StaffGateway --> API : Return success
API --> FE : Show SHIPPING

Staff -> FE : Click Mark Completed
FE -> API : PATCH /api/staff/orders/{orderId}/status
API -> StaffGateway : Validate Staff/Admin role

StaffGateway -> Order : REST update status to COMPLETED
Order -> DB : Check current status = SHIPPING
DB --> Order : Valid current status
Order -> DB : Update orderStatus = COMPLETED
Order -> DB : Insert order_status_history

Order -> Kafka : Publish OrderCompleted

Order --> StaffGateway : Return success
StaffGateway --> API : Return success
API --> FE : Show COMPLETED

Kafka -> Order : Event stored for other consumers

Kafka -> StaffGateway : Optional consume for notification or operation log

note right of StaffGateway

Staff Gateway checks role.

It must not update order table directly.

end note

note right of Order

Java Order Service owns order lifecycle.

REST is used because Staff needs update result immediately.

Kafka is only used after status changed.

end note

@enduml

Feature-Fin(Payment)

Payment Scope — MVP Provider-Ready Version

Payment Strategy

Trong MVP, Payment Service không chỉ là mock đơn giản. Hệ thống sẽ thiết kế theo hướng:

Mock-first, Provider-ready

Nghĩa là:

- Giai đoạn đầu dùng **MOCK** để demo flow nhanh.
 - Sau đó có thể gắn PayOS hoặc SePay mà không cần đổi kiến trúc lớn.
 - Payment Service vẫn thuộc C# .NET 8.
 - Order Service vẫn thuộc Java Spring Boot.
 - C# xử lý payment provider.
 - Java xử lý order lifecycle.
-

Payment Provider Options

Option 1: MOCK

Dùng cho demo nội bộ.

Flow:

OrderCreated

↓

C# Payment Service tạo payment

↓

Mock payment success

↓

PaymentSucceeded event

↓

Java Order Service cập nhật order PAID

Option 2: PayOS

Dùng khi muốn demo flow thanh toán gần thực tế.

PayOS phù hợp cho:

- Tạo payment link
- Redirect user sang trang thanh toán
- Nhận webhook payment result
- Cập nhật payment status
- Tự động phát event **PaymentSucceeded**

Flow:

```
graph TD
    A[Java Order Service tạo order] --> B[Publish OrderCreated]
    B --> C[C# Payment Service consume OrderCreated]
    C --> D[C# tạo payment link qua PayOS]
    D --> E[Frontend nhận checkout_url]
    E --> F[Customer thanh toán]
    F --> G[PayOS gọi webhook về C#]
    G --> H[C# verify webhook]
    H --> I[C# update payments = PAID]
    I --> J[C# publish PaymentSucceeded]
    J --> K[Java Order Service update order = PAID]
```

Option 3: SePay

Dùng khi muốn học flow QR chuyển khoản ngân hàng và webhook đối soát.

SePay phù hợp cho:

- Tạo QR chuyển khoản
- Nhận webhook khi có giao dịch ngân hàng
- Đối soát nội dung chuyển khoản
- Tự động xác nhận order đã thanh toán

Flow:

Java Order Service tạo order
↓
Publish OrderCreated
↓
C# Payment Service tạo payment record
↓
C# tạo QR hoặc payment instruction
↓
Customer quét QR và chuyển khoản
↓
SePay gửi webhook về C#
↓
C# kiểm tra amount/order_code
↓
C# update payments = PAID
↓
C# publish PaymentSucceeded
↓
Java Order Service update order = PAID

Recommended MVP Decision

Trong MVP learning version, chọn hướng sau:

Phase 1: MOCK

Phase 2: PayOS hoặc SePay

Nếu cần demo nhanh và rõ ràng:

Recommend: PayOS

Nếu muốn học sâu về chuyển khoản ngân hàng, webhook và đối soát:

Alternative: SePay

Updated payments Table

Table: payments

Owner

C# Payment Service

Purpose

Lưu thông tin thanh toán mock hoặc thanh toán qua PayOS/SePay.

Columns

Column	Type	Required	Description
id	UUID	Yes	Primary key
order_id	UUID	Yes	Order ID từ Java Order Service
customer_id	UUID	Yes	Customer ID
amount	DECIMAL(18,2)	Yes	Số tiền thanh toán
payment_method	VARCHAR(50)	Yes	MOCK, PAYOS, SEPAY
payment_status	VARCHAR(30)	Yes	PENDING, PAID, FAILED, CANCELLED
provider_payment_id	VARCHAR(255)	No	Mã payment từ PayOS/SePay
transaction_code	VARCHAR(100)	No	Mã giao dịch hoặc mã chuyển khoản
checkout_url	TEXT	No	Link thanh toán, dùng cho PayOS
qr_code_url	TEXT	No	QR thanh toán, dùng cho PayOS/SePay
payment_content	VARCHAR(255)	No	Nội dung chuyển khoản, ví dụ ORDER123
invoice_number	VARCHAR(100)	No	Mã hóa đơn
invoice_pdf_url	TEXT	No	URL file PDF hóa đơn

paid_at	TIMESTAMP	No	Thời điểm thanh toán thành công
created_at	TIMESTAMP	Yes	Ngày tạo
updated_at	TIMESTAMP	Yes	Ngày cập nhật

Payment Status

PENDING
PAID
FAILED
CANCELLED

Payment Method

MOCK
PAYOS
SEPAY

Payment Business Rules

ID	Rule
PAY-BR-001	Payment amount phải bằng order total amount
PAY-BR-002	Một order chỉ có một payment active tại một thời điểm
PAY-BR-003	Khi payment PAID, C# phải publish PaymentSucceeded event
PAY-BR-004	Java Order Service mới được update order status thành PAID
PAY-BR-005	C# Payment Service không update trực tiếp bảng orders
PAY-BR-006	Nếu payment FAILED hoặc CANCELLED, order vẫn ở PENDING_PAYMENT

PAY-BR-007	Với SePay, cần kiểm tra amount và payment_content trước khi xác nhận PAID
PAY-BR-008	Với PayOS, cần verify webhook trước khi xác nhận PAID
PAY-BR-009	Sau khi payment PAID, Java Order Service phải lock design
PAY-BR-010	Invoice PDF chỉ tạo sau khi payment PAID

C# Payment Service Responsibilities

C# Payment Service xử lý:

- Create payment record
 - Mock payment success/fail
 - Tích hợp PayOS hoặc SePay
 - Nhận webhook từ provider
 - Verify webhook
 - Update payments table
 - Generate invoice number
 - Generate invoice PDF
 - Store invoice PDF URL
 - Publish PaymentSucceeded event
-

Java Order Service Responsibilities

Java Order Service xử lý:

- Create order
 - Create order items
 - Calculate order total
 - Publish OrderCreated event
 - Consume PaymentSucceeded event
 - Update order payment_status = PAID
 - Update order_status = PAID
 - Lock design after payment
 - Insert order_status_history
-

Kafka Events

OrderCreated

Producer:

Java Order Service

Consumer:

C# Payment Service

Purpose:

Tạo payment record và bắt đầu payment flow

PaymentSucceeded

Producer:

C# Payment Service

Consumer:

Java Order Service

Purpose:

Cập nhật order đã thanh toán và lock design

PaymentFailed

Producer:

C# Payment Service

Consumer:

Java Order Service

Purpose:

Ghi nhận thanh toán thất bại

Final Payment Decision

MVP không ghi là “mock đơn giản” nữa.

Thay bằng:

Payment Service supports MOCK first, then can integrate PayOS or SePay using the same payments table and Kafka flow.

Không cần thêm bảng mới.

Chỉ cần mở rộng bảng **payments** bằng các cột:

provider_payment_id
checkout_url
qr_code_url
payment_content
invoice_number
invoice_pdf_url
updated_at

api

CORE API MAP & CORE PROJECT STRUCTURE

AI T-Shirt Customization & Virtual Try-On Platform

Version: Core MVP v2.0

Scope: E-commerce + T-shirt Customization + AI Try-On + Payment

Frontend: React + TypeScript

Backend Java: Core Business System

Backend C#: Platform / Payment / Invoice System

Database: PostgreSQL

Messaging: Kafka

1. Core Scope Update

Bản core mới không còn là bản lite 33 API nữa.

Core MVP cần có:

Category
Product
Cart
Design
AI Try-On
Order
Payment
Invoice
Invoice Items
Feedback
About Us
Staff/Admin Operation

Vì đây là hệ thống e-commerce, nên không thể thiếu:

categories
carts
cart_items
invoices
invoice_items

2. Final API Count

Type	Count
Frontend-facing REST APIs	50
Internal service-to-service APIs	2
Kafka Events	7
Core Database Tables	23

3. Backend Ownership Summary

Backend	API Count	Main Responsibility
C# .NET 8	21 APIs	Auth, User profile, Payment, Invoice, About Us, Staff Gateway
Java Spring Boot	29 APIs	Category, Product, Cart, Design, Try-On, Order, Feedback
Internal APIs	2 APIs	Service-to-service query
Total Frontend-facing APIs	50 APIs	Core MVP

4. Core Database Tables — 23 Tables

4.1 Identity Tables — C# Owns

No.	Table	Purpose
1	users	Lưu tài khoản
2	roles	Lưu role ADMIN, STAFF, CUSTOMER
3	user_roles	Gán role cho user

4	refresh_token s	Quản lý refresh token
---	--------------------	-----------------------

4.2 Catalog Tables — Java Owns

No.	Table	Purpose
5	categories	Lưu danh mục sản phẩm
6	products	Lưu sản phẩm áo
7	product_variants	Lưu size, màu, chất liệu, SKU
8	product_images	Lưu ảnh sản phẩm
9	product_inventory	Lưu tồn kho theo variant

4.3 Cart Tables — Java Owns

No.	Table	Purpose
10	carts	Lưu giỏ hàng của customer
11	cart_items	Lưu item trong giỏ hàng

4.4 Design Tables — Java Owns

No.	Table	Purpose
12	designs	Lưu thiết kế áo
13	design_layer s	Lưu layer trong design

4.5 AI Try-On Tables — Java Owns

No.	Table	Purpose
14	tryon_request s	Lưu request Try-On
15	tryon_results	Lưu kết quả Try-On

4.6 Order Tables — Java Owns

No.	Table	Purpose
16	orders	Lưu đơn hàng
17	order_items	Lưu sản phẩm trong đơn
18	order_status_history	Lưu lịch sử trạng thái đơn

4.7 Payment & Invoice Tables — C# Owns

No.	Table	Purpose
19	payments	Lưu payment transaction
20	invoices	Lưu hóa đơn
21	invoice_items	Lưu từng dòng item trong hóa đơn

4.8 Feedback & Content Tables

No.	Table	Owner	Purpose
22	feedbacks	Java	Lưu feedback
23	about_us_content	C#	Lưu nội dung About Us

5. C# APIs — 21 APIs

5.1 Authentication APIs — 5 APIs

No.	Method	Endpoint	Purpose
1	POST	/api/auth/register	Tạo tài khoản customer

2	POST	/api/auth/login	Đăng nhập
3	POST	/api/auth/refresh-token	Cấp access token mới
4	POST	/api/auth/logout	Đăng xuất
5	GET	/api/auth/me	Lấy thông tin user hiện tại

5.2 User Profile APIs — 3 APIs

No.	Method	Endpoint	Purpose
6	GET	/api/users/me/profile	Xem hồ sơ cá nhân
7	PATCH	/api/users/me/profile	Cập nhật hồ sơ cá nhân
8	POST	/api/users/me/avatar	Upload avatar

5.3 Payment APIs — 5 APIs

No.	Method	Endpoint	Purpose
9	POST	/api/payments	Tạo payment PayOS/SePay
10	GET	/api/payments/{paymentId}	Lấy chi tiết payment
11	GET	/api/payments/order/{orderId}	Lấy payment theo order
12	POST	/api/payments/webhooks/payos	Nhận webhook PayOS
13	POST	/api/payments/webhooks/sepay	Nhận webhook SePay

5.4 Invoice APIs — 4 APIs

No.	Method	Endpoint	Purpose
14	GET	/api/invoices/{invoiceId}	Lấy chi tiết invoice
15	GET	/api/invoices/order/{orderId}	Lấy invoice theo order
16	GET	/api/invoices/{invoiceId}/items	Lấy invoice items

17	GET	/api/invoices/{invoiceId}/pdf	Tải invoice PDF
----	-----	-------------------------------	-----------------

5.5 About Us APIs — 2 APIs

No.	Method	Endpoint	Purpose
18	GET	/api/about-us	Public xem About Us
19	PUT	/api/admin/about-us/{sectionKey}	Staff/Admin cập nhật About Us

5.6 Staff Gateway APIs — 2 APIs

No.	Method	Endpoint	Purpose
20	GET	/api/staff/orders	Staff xem danh sách đơn
21	GET	/api/staff/orders/{orderId}/print-info	Staff lấy file in

6. Java APIs — 29 APIs

6.1 Category APIs — 4 APIs

No.	Method	Endpoint	Purpose
22	GET	/api/categories	Xem danh sách category
23	GET	/api/categories/{categoryId}	Xem chi tiết category
24	POST	/api/admin/categories	Tạo category
25	PUT	/api/admin/categories/{categoryId}	Cập nhật category

6.2 Product APIs — 6 APIs

No.	Method	Endpoint	Purpose
	d		

26	GET	/api/products	Xem danh sách sản phẩm
27	GET	/api/products/{productId}	Xem chi tiết sản phẩm
28	POST	/api/admin/products	Tạo sản phẩm
29	PUT	/api/admin/products/{productId}	Cập nhật sản phẩm
30	POST	/api/admin/products/{productId}/images	Thêm ảnh sản phẩm
31	PATCH	/api/admin/product-variants/{variantId}/inventory	Cập nhật tồn kho

6.3 Cart APIs — 5 APIs

No.	Method	Endpoint	Purpose
32	GET	/api/cart	Lấy giỏ hàng hiện tại
33	POST	/api/cart/items	Thêm item vào giỏ
34	PATCH	/api/cart/items/{cartItemId}	Cập nhật số lượng item
35	DELETE	/api/cart/items/{cartItemId}	Xóa item khỏi giỏ
36	DELETE	/api/cart	Xóa toàn bộ giỏ hàng

6.4 Design APIs — 4 APIs

No.	Method	Endpoint	Purpose
37	POST	/api/designs	Tạo design draft
38	PUT	/api/designs/{designId}/save	Lưu thiết kế
39	GET	/api/designs/my	Xem thiết kế của tôi
40	GET	/api/designs/{designId}	Xem chi tiết design

6.5 AI Try-On APIs — 3 APIs

No.	Method	Endpoint	Purpose
41	POST	/api/tryon/requests	Tạo yêu cầu Try-On

42	GET	/api/tryon/requests/{requestId}	Kiểm tra trạng thái Try-On
43	GET	/api/tryon/requests/{requestId}/result	Lấy kết quả Try-On

6.6 Order APIs — 4 APIs

No.	Method	Endpoint	Purpose
44	POST	/api/orders	Tạo order từ cart hoặc design
45	GET	/api/orders/my	Customer xem order của mình
46	GET	/api/orders/{orderId}	Xem chi tiết order
47	PATCH	/api/staff/orders/{orderId}/status	Staff cập nhật trạng thái order

6.7 Feedback APIs — 3 APIs

No.	Method	Endpoint	Purpose
48	POST	/api/feedbacks	Customer gửi feedback
49	GET	/api/feedbacks/public	Xem feedback public
50	PATCH	/api/admin/feedbacks/{feedbackId}/status	Staff/Admin duyệt feedback

7. Internal APIs — 2 APIs

Internal APIs không cho frontend gọi.

No.	Method	Endpoint	Caller	Purpose
I-01	GET	/api/internal/orders/{orderId}/payment-info	C# Payment Service	Lấy order amount, customerId, orderStatus
I-02	GET	/api/internal/orders/{orderId}/invoice-snapshot	C# Invoice Service	Lấy order items để tạo invoice và invoice_items

8. Kafka Events — 7 Events

No	Event	Producer	Consumer	Purpose
1	OrderCreated	Java Order Service	C# Payment Service	Báo order mới được tạo
2	PaymentCreated	C# Payment Service	Java Order Service / Log later	Báo payment đã được tạo
3	PaymentSucceeded	C# Payment Service	Java Order Service	Báo payment thành công
4	PaymentFailed	C# Payment Service	Java Order Service	Báo payment thất bại
5	TryOnRequested	Java Try-On Service	AI Worker	Gửi job Try-On
6	TryOnCompleted	AI Worker	Java Try-On Service	Báo Try-On thành công
7	OrderCompleted	Java Order Service	Feedback / Notification later	Báo đơn hoàn tất

9. Important Core Business Rules

9.1 Category Rules

Rule	Description
CAT-BR-001	Product phải thuộc một category
CAT-BR-002	Category inactive thì product public không hiển thị
CAT-BR-003	Không xóa cứng category nếu đã có product

9.2 Cart Rules

Rule	Description
CART-BR-001	Customer có một active cart

CART-BR-002	Cart item phải gắn với product variant
CART-BR-003	Nếu item là áo custom thì phải có designId
CART-BR-004	Không cho add item nếu variant hết hàng
CART-BR-005	Khi tạo order thành công thì cart có thể clear hoặc mark checked out

9.3 Invoice Rules

Rule	Description
INV-BR-001	Invoice chỉ tạo sau payment PAID
INV-BR-002	Invoice phải có invoice_items
INV-BR-003	Invoice item snapshot lấy từ order_items
INV-BR-004	Invoice PDF sinh sau khi invoice được tạo
INV-BR-005	Customer chỉ xem invoice của order thuộc mình

9.4 Payment Rules

Rule	Description
PAY-BR-001	Payment amount phải bằng order totalAmount
PAY-BR-002	C# Payment Service owns payments
PAY-BR-003	Java Order Service owns orders
PAY-BR-004	C# không update trực tiếp order table
PAY-BR-005	Java chỉ update order sau khi consume PaymentSucceeded

10. Core Backend Java Folder Structure

Java là core business system.

```
backend-java/
src/main/java/com/aitshirt/core/
```

- common/
 - response/
 - exception/
 - security/
 - config/

- category/
 - controller/
 - CategoryController.java
 - AdminCategoryController.java
 - service/
 - CategoryService.java
 - repository/
 - CategoryRepository.java
 - entity/
 - Category.java
 - dto/
 - CategoryResponse.java
 - CreateCategoryRequest.java
 - UpdateCategoryRequest.java

- catalog/
 - controller/
 - ProductController.java
 - AdminProductController.java
 - service/
 - ProductService.java
 - ProductImageService.java
 - InventoryService.java
 - repository/
 - ProductRepository.java
 - ProductVariantRepository.java
 - ProductImageRepository.java
 - ProductInventoryRepository.java
 - entity/
 - Product.java
 - ProductVariant.java
 - ProductImage.java
 - ProductInventory.java
 - dto/
 - ProductListResponse.java
 - ProductDetailResponse.java
 - CreateProductRequest.java
 - UpdateProductRequest.java
 - UpdateInventoryRequest.java

- cart/

- controller/
 - CartController.java
- service/
 - CartService.java
- repository/
 - CartRepository.java
 - CartItemRepository.java
- entity/
 - Cart.java
 - CartItem.java
- dto/
 - CartResponse.java
 - AddCartItemRequest.java
 - UpdateCartItemRequest.java

- design/
 - controller/
 - DesignController.java
 - service/
 - DesignService.java
 - repository/
 - DesignRepository.java
 - DesignLayerRepository.java
 - entity/
 - Design.java
 - DesignLayer.java
 - dto/
 - CreateDesignRequest.java
 - SaveDesignRequest.java
 - DesignDetailResponse.java

- tryon/
 - controller/
 - TryOnController.java
 - service/
 - TryOnService.java
 - kafka/
 - TryOnRequestedProducer.java
 - TryOnCompletedConsumer.java
 - repository/
 - TryOnRequestRepository.java
 - TryOnResultRepository.java
 - entity/
 - TryOnRequest.java
 - TryOnResult.java
 - dto/
 - CreateTryOnRequest.java
 - TryOnStatusResponse.java

TryOnResultResponse.java

order/

controller/

OrderController.java

StaffOrderController.java

InternalOrderController.java

service/

OrderService.java

OrderStatusService.java

OrderSnapshotService.java

kafka/

OrderCreatedProducer.java

PaymentSucceededConsumer.java

PaymentFailedConsumer.java

OrderCompletedProducer.java

repository/

OrderRepository.java

OrderItemRepository.java

OrderStatusHistoryRepository.java

entity/

Order.java

OrderItem.java

OrderStatusHistory.java

dto/

CreateOrderRequest.java

OrderDetailResponse.java

OrderPaymentInfoResponse.java

OrderInvoiceSnapshotResponse.java

UpdateOrderStatusRequest.java

feedback/

controller/

FeedbackController.java

AdminFeedbackController.java

service/

FeedbackService.java

repository/

FeedbackRepository.java

entity/

Feedback.java

dto/

SubmitFeedbackRequest.java

FeedbackResponse.java

ModerateFeedbackRequest.java

11. Core Backend C# Folder Structure

C# là platform, payment, invoice system.

backend-csharp/

src/

Al.Tshirt.Api/

Controllers/

AuthController.cs

UserProfileController.cs

PaymentController.cs

PaymentWebhookController.cs

InvoiceController.cs

AboutUsController.cs

StaffGatewayController.cs

Middlewares/

ExceptionMiddleware.cs

RequestLoggingMiddleware.cs

Al.Tshirt.Application/

Auth/

Services/

AuthService.cs

TokenService.cs

DTOs/

AuthResponse.cs

CurrentUserResponse.cs

Users/

Services/

UserProfileService.cs

DTOs/

UserProfileResponse.cs

UpdateProfileRequest.cs

Payments/

Services/

PaymentService.cs

Providers/

IPaymentProvider.cs

PayOSProvider.cs

SePayProvider.cs

Webhooks/

PayOSWebhookHandler.cs

SePayWebhookHandler.cs

DTOs/

- CreatePaymentRequest.cs
- PaymentResponse.cs
- PaymentStatusResponse.cs

- Invoices/

- Services/

- InvoiceService.cs

- InvoicePdfService.cs

- DTOs/

- InvoiceResponse.cs

- InvoiceItemResponse.cs

- InvoicePdfResponse.cs

- AboutUs/

- Services/

- AboutUsService.cs

- DTOs/

- AboutUsResponse.cs

- UpsertAboutUsRequest.cs

- StaffGateway/

- Services/

- StaffGatewayService.cs

- DTOs/

- StaffOrderResponse.cs

- PrintInfoResponse.cs

- Common/

- Responses/

- Exceptions/

- Al.Tshirt.Domain/

- Entities/

- User.cs

- Role.cs

- UserRole.cs

- RefreshToken.cs

- Payment.cs

- Invoice.cs

- InvoiceItem.cs

- AboutUsContent.cs

- Enums/

- UserStatus.cs

- PaymentStatus.cs

- PaymentMethod.cs

- InvoiceStatus.cs

- RoleName.cs

Al.Tshirt.Infrastructure/
Database/
AppDbContext.cs

Repositories/
UserRepository.cs
RoleRepository.cs
RefreshTokenRepository.cs
PaymentRepository.cs
InvoiceRepository.cs
InvoiceItemRepository.cs
AboutUsRepository.cs

Kafka/
OrderCreatedConsumer.cs
PaymentCreatedProducer.cs
PaymentSucceededProducer.cs
PaymentFailedProducer.cs

HttpClients/
JavaOrderClient.cs
JavaDesignClient.cs
JavaFeedbackClient.cs

Storage/
IStorageService.cs
LocalStorageService.cs
BlobStorageService.cs

12. Core Frontend Folder Structure

Frontend vẫn giữ ở mức fresher nhưng có đủ core ecommerce flow.

```
frontend/  
  src/  
    app/  
      App.tsx  
      router.tsx  
      providers.tsx  
  
  shared/  
    api/  
      httpClient.ts
```

- apiResponse.ts
- components/
 - Button.tsx
 - Input.tsx
 - Textarea.tsx
 - Select.tsx
 - Card.tsx
 - Badge.tsx
 - Modal.tsx
 - Loading.tsx
 - EmptyState.tsx
 - ErrorState.tsx
 - ProtectedRoute.tsx
 - RoleGuard.tsx
- constants/
 - roles.ts
 - routes.ts
 - orderStatus.ts
 - paymentStatus.ts
- utils/
 - formatCurrency.ts
 - formatDate.ts

- features/
 - auth/
 - api.ts
 - types.ts
 - authStore.ts
 - LoginPage.tsx
 - RegisterPage.tsx

- user/
 - api.ts
 - types.ts
 - ProfilePage.tsx
 - EditProfilePage.tsx

- category/
 - api.ts
 - types.ts
 - CategoryList.tsx
 - CategoryManagementPage.tsx

- catalog/
 - api.ts
 - types.ts
 - ProductListPage.tsx
 - ProductDetailPage.tsx

ProductCard.tsx
ProductManagementPage.tsx
InventoryPage.tsx

cart/

api.ts
types.ts
CartPage.tsx
CartItemCard.tsx
CartSummary.tsx

design/

api.ts
types.ts
designStore.ts
MyDesignsPage.tsx
DesignEditorPage.tsx
TShirtCanvas.tsx
EditorToolbar.tsx

tryon/

api.ts
types.ts
TryOnPage.tsx
TryOnResultPage.tsx

order/

api.ts
types.ts
CheckoutPage.tsx
MyOrdersPage.tsx
OrderDetailPage.tsx

payment/

api.ts
types.ts
PaymentPage.tsx
PaymentSuccessPage.tsx
PaymentCancelPage.tsx
PayOSBox.tsx
SePayBox.tsx

invoice/

api.ts
types.ts
InvoiceDetailPage.tsx
InvoiceDownloadButton.tsx

feedback/
api.ts
types.ts
SubmitFeedbackPage.tsx
PublicFeedbackList.tsx
FeedbackModerationPage.tsx

about/
api.ts
types.ts
AboutUsPage.tsx
AboutUsManagementPage.tsx

staff/
api.ts
types.ts
StaffDashboardPage.tsx
StaffOrderListPage.tsx
StaffOrderDetailPage.tsx
PrintInfoPage.tsx

admin/
AdminDashboardPage.tsx

13. Final Core Decision

Core MVP mới nên chốt:

50 frontend-facing REST APIs
2 internal REST APIs
7 Kafka events
23 core database tables

Backend chia rõ:

Java = Core Business System
C# = Platform / Payment / Invoice System

Java owns:

Category
Product
Cart

Design
Try-On
Order
Feedback

C# owns:

Auth
User Profile
Payment
Invoice
Invoice Items
About Us
Staff Gateway

Kafka dùng cho:

Try-On async
Payment success/failure
Order lifecycle events

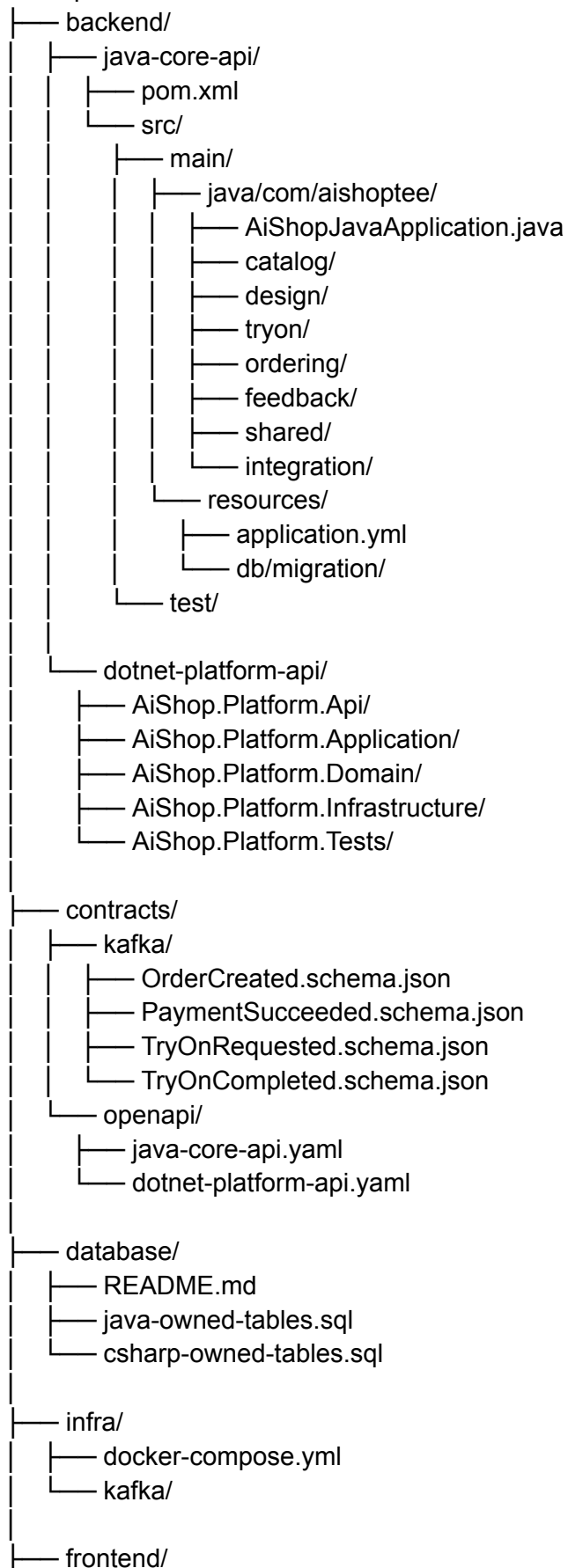
REST dùng cho:

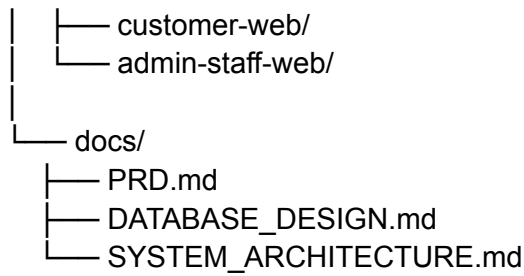
User action cần phản hồi ngay
Login
Product
Cart
Design save
Create order
Create payment
Staff update order

Cấu trúc folder

Cấu trúc đề xuất:

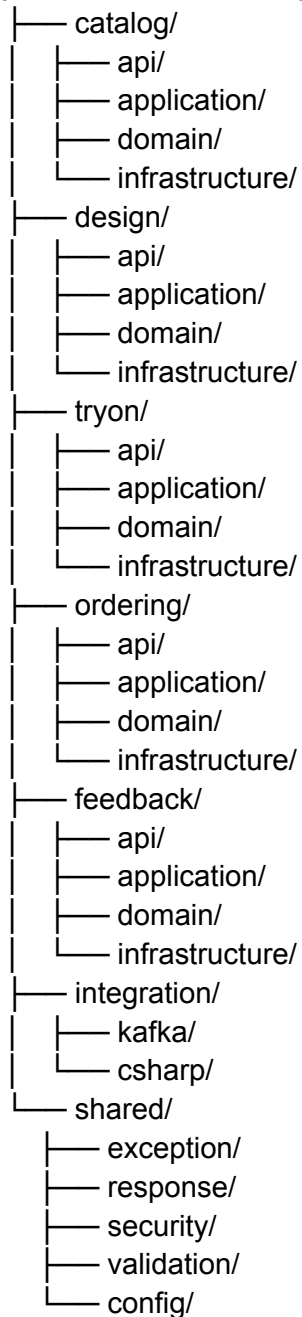
AIShopTee3d/





Java bên bạn nên chia package như này:

java-core-api/src/main/java/com/aishoptee/



Ý nghĩa:

api Controller, request/response DTO

application Use case, command/query, business orchestration

domain Entity, enum, rule, domain service, repository interface

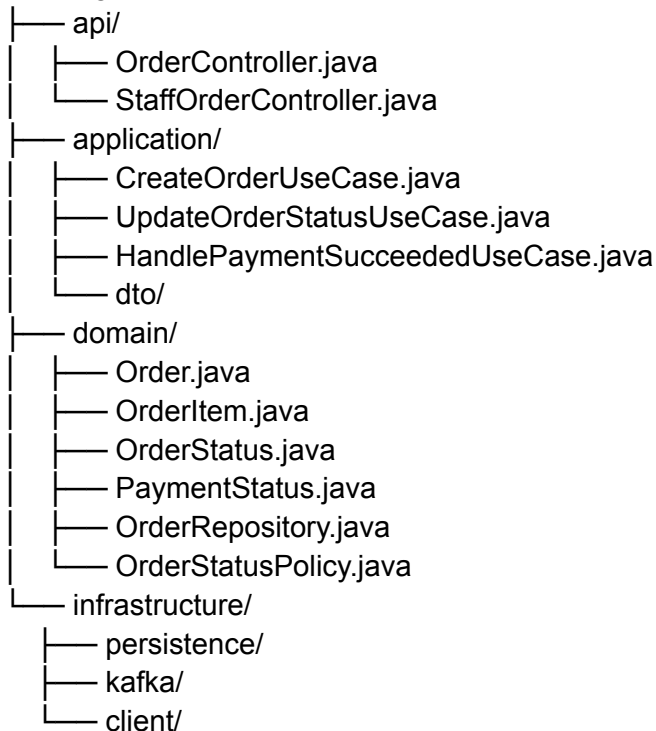
infrastructure JPA entity, repository impl, Kafka adapter, external client

shared Common exception, response format, config

integration Kafka events + gọi API C#

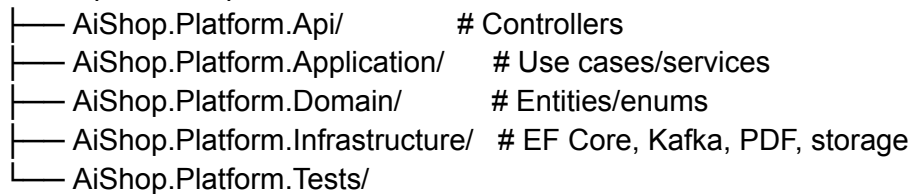
Ví dụ module ordering:

ordering/



Với C# backend, người làm C# nên giữ riêng:

dotnet-platform-api/



C# owns:

identity

payment

content

operation gateway

Java owns:

catalog

design

tryon

ordering

feedback

env


```
POSTGRES_DB=ai_fashion_studio_db
POSTGRES_USER=admin
POSTGRES_PASSWORD=admin_db@@
POSTGRES_PORT=15432

PGADMIN_EMAIL=admin@example.com
PGADMIN_PASSWORD=admin_db@@
PGADMIN_PORT=15050

KAFKA_INTERNAL_PORT=9092
KAFKA_EXTERNAL_PORT=39092
KAFKA_UI_PORT=18085
KAFKA_IMAGE=apache/kafka:3.7.0

MINIO_ROOT_USER=admin
MINIO_ROOT_PASSWORD=admin_db@@
MINIO_API_PORT=19000
MINIO_CONSOLE_PORT=19001
```

Main secret main pgadmin = PGADMIN_PASSWORD_AIFASHIONSTUDIO

convention

main

Nhánh chính, luôn ổn định, dùng để deploy/release.

develop

Nhánh tích hợp dev, gom code từ các feature/fix trước khi merge lên main.

feature/*

Làm tính năng mới.

fix/*

Sửa bug.

chore/*

Việc config, docker, dependency, build.

docs/*

Tài liệu.

refactor/*

Sửa cấu trúc code không đổi behavior.

Naming nên dùng lowercase + kebab-case:

feature/catalog-create-api

fix/swagger-gateway-catalog-route

chore/springdoc-gateway-config

hotfix/payment-build-failure

Commit message dùng Conventional Commits:

feat(catalog): add create catalog endpoint

fix(gateway): route catalog docs through swagger ui

fix(platform): add payment order domain model

chore(springdoc): align springdoc version with spring boot

docs(api): document swagger endpoints